

Road Trip Cost Prediction

Predicting the cost of a road trip between any two Indian cities — and being honest about what the model actually knows.

Author	Param Kotadiya
Subject	Machine Learning (Semester 5)
Artifact	RoadTripCost.ipynb — Weeks 1–10, 162 cells, all executed
Problem type	Regression (primary) + 2 classification tasks
Target	total_trip_cost (continuous, ₹)
Datasets	1,140 original trips · 25,000 generated trips · 840 real road distances
Models trained	9 sub-models / architectures, 7 algorithm families each
Tests	65 automated tests, all passing

The one thing to remember before a viva

This project reports **two** accuracy figures, and knowing why is the whole point of it.

$R^2 = 0.999707$ is real, but it was measured with the true `toll_cost`, `parking_cost` and `fuel_consumption_litres` handed to the model as input features. Those are three of the four terms of the cost formula, and a traveller knows none of them before setting off. It answers “*given the litres burnt and the toll paid, can you total the bill?*”

$R^2 = 0.9645$, **MAE ₹391** is what the model achieves predicting from what a user actually knows: two city names, a vehicle, a fuel type and a departure time. That is the number the deployed application reports, and the honest one to quote.

CONTENTS

What is in this report

This document is written to be **read once, cover to cover, before a viva**. It covers every week of the project, every model that was trained (including the ones that failed), every number that appears in the notebook, and the reasoning behind each decision. Part 15 turns all of it into likely examiner questions with prepared answers.

Part 1	The project at a glance — problem, data, stack, file map	overview
Part 2	The dataset in detail — every column, every statistic	data
Part 3	Week 1 — Problem definition and dataset exploration	week 1
Part 4	Week 2 — Data preprocessing, all 8 steps	week 2
Part 5	Week 3 — Linear regression: <code>fit()</code> and gradient descent from scratch	week 3
Part 6	Week 4 — Proving the dataset is synthetic; recovering the formula	week 4
Part 7	Week 5 — Scaling from 9 routes to 534 cities	week 5
Part 8	Week 6 — One engineered feature beats two ensembles	week 6
Part 9	Week 7 — Classification, and which target is real	week 7
Part 10	Week 8 — Why gradient descent stopped converging	week 8
Part 11	Week 9 — Task 5: evaluation, validation, tuning	week 9
Part 12	Week 10 — Removing the magic numbers: a chained pipeline	week 10
Part 13	The application — API, frontend, deployment, tests	engineering
Part 14	Bugs found in this project's own code, and limitations	honesty
Part 15	Viva preparation — 60 questions with answers	Q&A
Part 16	Glossary of every technical term used	reference
Part 17	How to reproduce every number in this report	reference

A note on how to read the tables

Wherever a model is scored, the report gives **train** and **test** figures side by side, plus a **cross-validation mean and standard deviation**. A single test score can be luck; the CV standard deviation is what says whether a model is stable. Wherever a constant was replaced by a model, the constant is kept in the table as the **baseline row** — a model that cannot beat the thing it replaced has not earned its place.

The project at a glance

1.1 The problem

Road trips involve several variable costs — fuel, tolls, parking, wear on the vehicle — which depend on the route, the vehicle, traffic conditions and how far you are driving. Estimating that total in advance is useful for planning and budgeting.

Item	Value
Problem statement	Given the details of a road trip, predict the total trip cost using machine learning.
Target variable	<code>total_trip_cost</code> — continuous, in rupees
Problem type	Regression (a continuous numeric value). Two classification tasks were added in Week 7 where they add value.
Original dataset	<code>road_trip_data.csv</code> — 1,140 rows, 16 columns, 9 fixed city pairs
Final serving domain	Any two of 537 Indian cities (population $\geq$ 100,000), 50–1,500 km

1.2 What the finished thing does

The user picks two cities, a vehicle, a fuel type and a departure time. They are asked only for things they could actually know. Everything else — road distance, their car's fuel efficiency, litres burnt, toll, traffic level — is **predicted by its own fitted sub-model**, and a chained cost regressor totals them. The response carries a full cost breakdown, an efficiency band, an honest error bar, and the caveats that apply to that particular prediction.

1.3 The ten weeks

Week	What it established	Headline result
1	Problem definition, dataset exploration, correlation analysis	1,140×16, no missing values, no duplicates
2	Preprocessing: 8-step checklist, outliers, encoding, scaling, 7 visualisations	1,140 → 1,129 rows; 16 → 31 columns
3	Linear regression by <code>sklearn.fit()</code> and by gradient descent written from scratch in NumPy	Both agree to 0.03% ; R^2 0.992841
4	Discovered the dataset is synthetic and recovered the formula that generated it	Reconstruction R^2 0.999162 from arithmetic alone
5	Regenerated the same cost physics over real geography for 534 cities	25,000 trips, 50–1,500 km
6	One engineered feature (<code>litres × price</code>) versus two ensemble models	MAE ₹143.85 → ₹39.66
7	Classification on two targets, with an explicit judgement on which is real	traffic 53.9% / cost-band 85.8%
8	Diagnosed why gradient descent stopped converging; rejected Ridge with evidence	$\kappa \approx 16,000$; ships OLS
9	Task 5 — metrics, fit diagnosis, cross-validation, bootstrap, comparison, tuning, ensembles	2 of 3 tuning searches improved
10	Replaced 5 hardcoded constants with 6 fitted sub-models; measured the honest accuracy	R^2 0.9645 , MAE ₹391

1.4 Technology used

Layer	Choice	Why
Language	Python 3.13	Standard for ML coursework
Data	pandas 3.0.3, NumPy	Loading, cleaning, tabular manipulation
Models	scikit-learn 1.9.0	LinearRegression, Ridge, DecisionTree, RandomForest, AdaBoost, GradientBoosting, LogisticRegression
From scratch	NumPy only	Gradient descent, so Week 3 shows what <code>fit()</code> does underneath
Charts	matplotlib, seaborn	All notebook visualisations
Backend	FastAPI + uvicorn	Serves the models over HTTP
Frontend	React 19, Vite 7, Tailwind v4, GSAP	Single-page editorial site
Routing data	OSRM public API	Real road distances
Geography	GeoNames <code>cities15000</code> (CC BY 4.0)	3,739 Indian cities with real latitude/longitude
Testing	pytest	65 API and model-behaviour tests
Deployment	Docker, Render blueprint	One container runs API + site

1.5 File map

RoadTripCost.ipynb	Weeks 1-10. The graded artifact. 162 cells, all executed.
TASK5.md	Task 5 answered, tables filled (generated from JSON)
FINDINGS.md	Evidence log, 15 sections - including the dead ends
README.md	Front door: results, how to run, limitations
road_trip_data.csv	The original 1,140 trips. Never modified.
roadtrip_features.py	Feature builders shared by the scripts AND the API
app.py	FastAPI backend - the chained prediction pipeline
data/	
india_cities.csv	3,739 cities, real coordinates (GeoNames)
fuel_prices.csv	35 states; a `source` column marks verified vs estimated
road_trip_wide.csv	25,000 generated trips over 534 cities
real_distances.csv	840 REAL OSRM road distances - the only observed data
pipeline_report.json	Saved metrics for every sub-model
end_to_end_report.json	Saved metrics for the benchmark model
task5_evaluation.json	Every Task 5 number
models/	
model.joblib	Week 6 regressor + cost-band classifier + GD history (10 KB)
pipeline.joblib	The six sub-models + chained cost regressor (5.1 MB, served)
scripts/	
evaluation.py	The Task 5 checklist, implemented once
fetch_real_distances.py	Collects the 840 real road distances from OSRM
train_pipeline.py	Fits and scores the six sub-models
train_end_to_end.py	The honesty benchmark
evaluate_models.py	Runs the whole Task 5 sweep
build_task5_doc.py	Generates TASK5.md from the JSON
build_*_chapters.py	Generates notebook Weeks 4-8 and 9-10
execute_new_chapters.py	Executes only the appended cells
frontend/	
static/	Built frontend, served by FastAPI
tests/test_api.py	65 tests

Why there is a shared `roadtrip_features.py` at the project root

Every model is trained in `scripts/` and served from `app.py`. If those two built their feature vectors independently they would drift — a column reordered in one place and not the other gives predictions that are wrong but not *obviously* wrong. So each model has exactly one function that returns its features in a fixed order, and both sides call it. It sits at the project root rather than in `scripts/` because the Docker image excludes `scripts/`, so the API could not import from there.

The dataset in detail

2.1 Every column of the original dataset

road_trip_data.csv — 1,140 rows × 16 columns, 181 KB in memory. Eight float64, two int64, six text columns.

Column	Type	Meaning	Corr. with target
start_city	str	Origin city. Only 9 fixed pairs exist in this file.	–
destination_city	str	Destination city.	–
distance_km	float	Road distance of the trip.	+0.8315
vehicle_type	str	Hatchback, Sedan or SUV.	–
fuel_type	str	Petrol, Diesel or CNG.	–
mileage	float	Vehicle fuel efficiency, km per litre.	–0.3802
fuel_price	float	Price per litre on the day of travel.	+0.1450
passengers	int	Number of people on the trip, 1–5.	–0.0468
departure_hour	int	Hour of departure, 0–23.	–0.0261
traffic_level	str	Low, Medium or High.	–
weather_condition	str	Clear, Cloudy, Fog or Rain.	–
toll_cost	float	Tolls paid. A direct additive component of the bill.	+0.6701
parking_cost	float	Parking paid. Another direct additive component.	+0.0970
estimated_time_minutes	float	Journey duration.	+0.7282
fuel_consumption_litres	float	Litres actually burnt. The single strongest driver.	+0.9606
total_trip_cost	float	TARGET — the total bill in rupees.	1.0000

2.2 Descriptive statistics

Column	mean	std	min	25%	50%	75%	max
distance_km	239.77	85.06	110.13	158.46	263.48	286.07	459.64
mileage	15.45	3.27	8.18	12.98	15.47	17.91	23.12
fuel_price	93.04	8.96	72.13	88.09	94.79	98.92	107.99
passengers	3.07	1.41	1	2	3	4	5
departure_hour	11.58	6.87	0	6	12	18	23
toll_cost	303.64	116.54	99.47	215.75	289.57	381.89	665.22
parking_cost	68.86	45.32	0.00	40.00	60.00	120.00	150.00
estimated_time_minutes	246.83	67.95	95.63	204.00	254.25	291.66	—

Target range across the cleaned 1,129 rows: ₹688.79 to ₹4,271.82.

2.3 Categorical values, and the 9 fixed routes

Column	Distinct values
vehicle_type	Sedan, Hatchback, SUV — 3 values
fuel_type	Petrol, Diesel, CNG — 3 values
traffic_level	Low, Medium, High — 3 values
weather_condition	Clear, Cloudy, Fog, Rain — 4 values

The nine routes, with their straight-line distance and the actual road distance — this table becomes important in Weeks 5 and 10:

Route	Straight line (km)	Road (km)	Winding factor
Ahmedabad → Surat	205.0	274.6	1.340
Ahmedabad → Udaipur	207.9	271.3	1.305
Ahmedabad → Vadodara	102.9	116.9	1.136
Bengaluru → Mysuru	127.8	150.9	1.180
Delhi → Jaipur	239.3	288.9	1.207
Pune → Mumbai	119.5	161.2	1.349
Rajkot → Ahmedabad	201.4	223.7	1.111
Surat → Mumbai	236.1	302.4	1.280
Vadodara → Mumbai	360.4	436.1	1.210
Winding factor across these 9 routes			mean 1.2355, sd 0.0820

2.4 The three datasets, and which one is real

Dataset	Rows	What it is	Real or generated?
road_trip_data.csv	1,140	The dataset handed out with the assignment. 9 city pairs, 110–460 km.	Synthetic (proved in Week 4)
road_trip_wide.csv	25,000	The same cost physics regenerated over real Indian geography, 534 cities, 50–1,500 km.	Generated (Week 5), on real coordinates
real_distances.csv	840	Actual driving distances from the OSRM routing engine for a stratified sample of city pairs.	Observed — the only real data

Be ready for this question

"If your data is synthetic, what have you actually learned?"

Three answers, all defensible. First, the synthetic nature was **discovered, not assumed** — Week 4 recovered the generating formula from the data with R^2 0.999162, which is itself a result. Second, the geography is **real**: cities, coordinates and 840 road distances come from GeoNames and OSRM, so the distance sub-model is trained on observed data. Third, the project states explicitly which numbers are measurements of real skill and which are measurements of formula recovery — that distinction is Part 12 of this report and the reason two accuracy figures are quoted rather than one.

Problem definition and dataset exploration

3.1 What Week 1 did

1. Loaded `road_trip_data.csv` and inspected the first rows.
2. Stated the problem and identified the target as continuous → a regression problem.
3. Produced `df.shape`, `df.info()` and `df.describe()`.
4. Checked for missing values, duplicates, and listed the distinct value of every categorical column.

3.2 Findings

Check	Result
Shape	1,140 rows × 16 columns
Missing values	Zero in every one of the 16 columns (0.0%)
Duplicate rows	Zero
Data types	8 float64, 2 int64, 6 text — all already correct
Target	<code>total_trip_cost</code> , continuous → regression

Why “no missing values” was itself a clue

A real-world dataset of 1,140 road trips with *zero* missing values, zero duplicates and perfectly consistent dtypes is unusual. At the time this was recorded as a convenience. In Week 4 it became evidence: datasets this clean are usually generated. Noticing it early and testing it later is the honest version of the story.

3.3 The correlation analysis that drove feature selection

Every numeric column was correlated against the target. This single table decided which features Week 3 would use.

Feature	Pearson r	Interpretation
fuel_consumption_litres	+0.9606	Dominant. Burning fuel is the main cost.
distance_km	+0.8315	Strong. Longer trip → more fuel.
estimated_time_minutes	+0.7282	Strong, but nearly a copy of distance.
toll_cost	+0.6701	A direct additive component of the bill.
fuel_price	+0.1450	Weak alone — it only matters <i>multiplied</i> by litres. Week 6 exploits this.
parking_cost	+0.0970	Additive but small and random.
departure_hour	-0.0261	Effectively zero.
passengers	-0.0468	Effectively zero.
mileage	-0.3802	The only meaningful negative — an efficient car costs less to run.

The insight hiding in this table

fuel_price correlates only **+0.145** with cost, which looks unimportant. But fuel is the largest single cost, and the real fuel bill is $\text{litres} \times \text{price}$. Price appears weak on its own because it only matters *as a product* with litres — and a linear model can only add its features, never multiply them. Week 6 turns this observation into the project's biggest single accuracy gain.

Data preprocessing — all eight steps

Week 2 followed the standard 8-step preprocessing checklist. Each step is given below with what was done, why, and what changed.

Step 1 — Understand the data

Re-ran `shape`, `head()` and `info()` to confirm the starting point: 1,140×16, no nulls.

Step 2 — Handle missing values

Counted nulls per column both as a count and as a percentage. **Result: zero missing in all 16 columns.** No imputation was needed, so none was invented.

What would have been done otherwise: numeric columns → median (robust to outliers, unlike the mean); categorical columns → mode; a column missing more than about half its values → drop it rather than fabricate it. Being able to state the policy you *would* have used matters more in a viva than the fact that this dataset needed none.

Step 3 — Remove duplicate rows

`df.duplicated().sum()` = 0 before and after `drop_duplicates()`. Shape unchanged at 1,140×16.

Step 4 — Fix data types

Printed `df.dtypes` and explicitly searched for columns stored as text that ought to be numeric or dates. **Result: none.** The six text columns are genuinely categorical.

Step 5 — Handle outliers

Boxplots were drawn for all ten numeric columns, then the **IQR rule** was applied to each in turn:

$$IQR = Q3 - Q1 \quad \cdot \quad \text{keep rows where } Q1 - 1.5 \times IQR \leq x \leq Q3 + 1.5 \times IQR$$

Measure	Value
Rows before outlier removal	1,140
Rows after outlier removal	1,129
Rows dropped	11 (0.96%)

Boxplots were redrawn afterwards to confirm the extreme values were gone. Losing 11 rows out of 1,140 is a cheap price for removing points that would otherwise drag a least-squares fit.

An honest caveat on this step

The filter is applied column by column in a loop, so a row is dropped if it is extreme on *any* single column. That is stricter than a multivariate outlier test and it is applied **before** the train/test split, which technically lets information about the test rows influence which rows exist. With 11 rows out of 1,140 the effect is negligible, but the correct order in a strict pipeline is split first, then fit any data-dependent transformation on the training half only. Weeks 9 and 10 do exactly that for scaling — see Part 14.

Step 6 — Encode categorical columns

A model needs numbers. The six text columns were **one-hot encoded** with `pd.get_dummies(..., drop_first=True)`.

Item	Value
Columns encoded	<code>start_city</code> , <code>destination_city</code> , <code>vehicle_type</code> , <code>fuel_type</code> , <code>traffic_level</code> , <code>weather_condition</code>
Shape after encoding	1,129 rows × 31 columns (from 16)
Method	One-hot, because none of these categories is <i>ordinal</i>

Why one-hot and not label encoding. Label encoding would map Hatchback→0, Sedan→1, SUV→2, which tells the model that an SUV is “twice” a Sedan and that Sedan sits exactly between the other two. None of that is true. One-hot gives each category its own independent column and asserts no ordering. `drop_first=True` removes one column per group to avoid the *dummy variable trap* — if you keep all three vehicle columns they sum to 1 in every row, which is perfectly collinear with the intercept.

Step 7 — Scale numeric columns

`distance_km` runs into the hundreds while `passengers` is 1–5. Features on wildly different scales distort distance-based and gradient-based methods, so **MinMaxScaler** mapped every numeric feature into 0–1:

$$x' = (x - x_{min}) / (x_{max} - x_{min})$$

Nine columns were scaled: `distance_km`, `mileage`, `fuel_price`, `passengers`, `departure_hour`, `toll_cost`, `parking_cost`, `estimated_time_minutes`, `fuel_consumption_litres`.

One deliberate decision worth quoting

The **target was left unscaled**. That means every learned coefficient reads directly as “rupees of cost per unit of this feature”, and because the features are all on 0–1, a weight is literally “how many rupees the cost changes as this feature moves from its minimum to its maximum”. Scaling the target would have made the model equally accurate and completely unreadable.

Why MinMax and not StandardScaler? Standardisation (subtract mean, divide by standard deviation) assumes a roughly Gaussian spread and produces unbounded values. MinMax gives a bounded 0–1 range, which suits gradient descent — all partial derivatives end up on a comparable scale, so a single learning rate works for every weight.

Step 8 — Visualise

Chart	Column	What it showed
Histogram	<code>total_trip_cost</code>	Distribution of the target — roughly unimodal, right-leaning
Box plot	<code>distance_km</code>	Median, quartiles and remaining spread after cleaning
Scatter plot	distance vs cost	The clear positive relationship the model exploits
Line plot	cost over row order	A proxy for trend over time; no drift visible
Bar chart	<code>vehicle_type</code>	Trip counts per vehicle
Pie chart	<code>fuel_type</code>	Share of each fuel
Count plot	<code>traffic_level</code>	Class balance for the Week 7 classifier
Boxplot grid ×2	all 10 numeric	Before and after outlier removal

Week 2 output

The chain **cleaned** → **de-duplicated** → **outlier-trimmed** → **one-hot encoded** → **MinMax-scaled** produced `df_scaled`: 1,129 rows × 31 columns, which is where Week 3 begins.

Linear regression: `fit()` and gradient descent from scratch

Week 3 has two halves that must arrive at the same answer.

Part	What it does	Tool
A	Fit a linear model the fast, production way	<code>sklearn.linear_model.LinearRegression().fit()</code>
B	Find the <i>same</i> parameters by hand, to see what <code>fit()</code> does underneath	NumPy only — own loss, gradients, update rule

5.1 The model

Because several columns are useful, this is **multiple** linear regression:

$$\hat{Y} = w_1X_1 + w_2X_2 + \dots + w_kX_k + c$$

5.2 Feature selection — and the one decision that needs defending

Six numeric features were kept:

Feature	Why it is in
<code>distance_km</code>	Longer trip → more fuel burnt ($r = +0.83$)
<code>mileage</code>	A more efficient vehicle burns less ($r = -0.38$, the only strong negative)
<code>fuel_price</code>	Price per litre on the day of travel
<code>toll_cost</code>	A direct additive component ($r = +0.67$)
<code>parking_cost</code>	Another direct additive component
<code>fuel_consumption_litres</code>	Litres actually burnt — the strongest driver ($r = +0.96$)

Three groups were deliberately left out, each for a different reason:

Dropped	Reason
<code>passengers</code> , <code>departure_hour</code>	Correlation with the target is ≈ 0 (-0.05 and -0.03). They add noise, not signal.
<code>estimated_time_minutes</code>	Almost a copy of <code>distance_km</code> — it contributes multicollinearity and no new information.
All one-hot columns	Rank deficiency — see the box below.

Why dropping the one-hot columns was necessary, not lazy

In this dataset every route is a *fixed pair*. Delhi only ever goes to Jaipur, so the column `start_city_Delhi` and the column `destination_city_Jaipur` contain **identical values in every row**.

That makes the design matrix **rank-deficient**: infinitely many different weight combinations produce the exact same predictions. sklearn silently picks one of them using the pseudo-inverse; gradient descent, starting from zeros and walking downhill, would converge on a *different* one. Both would be equally correct and the two weight vectors would look completely different.

Since the entire point of Week 3 is to compare the two sets of parameters, keeping those columns would have made a correct result *look* broken. Dropping them keeps the comparison meaningful. This is the single most likely thing to be probed in a viva, and the answer is “identical columns → infinitely many solutions → the comparison becomes meaningless”.

Final matrices: `x` shape (1129, 6), values 0.000–1.000; `y` shape (1129,), ₹688.79–₹4,271.82.

5.3 Part A — sklearn

sklearn solves the **normal equations** internally, in one algebraic step:

$$w = (X^T X)^{-1} X^T y$$

Feature	Learned weight (₹)	Reading
<code>distance_km</code>	+225.04	Cost rises with distance
<code>mileage</code>	-79.41	Negative — efficiency saves money
<code>fuel_price</code>	+636.32	Dearer fuel, dearer trip
<code>toll_cost</code>	+547.35	Additive component
<code>parking_cost</code>	+165.49	Additive component
<code>fuel_consumption_litres</code>	+2938.49	Dominant — burning fuel is the main cost
<i>intercept (c)</i>	+301.32	Baseline cost

The learned equation:

```
total_trip_cost = (225.04 x distance_km) + (-79.41 x mileage) + (636.32 x fuel_price)
                  + (547.35 x toll_cost) + (165.49 x parking_cost)
                  + (2938.49 x fuel_consumption_litres) + 301.32
```

Because every feature was scaled to 0–1, a weight is directly comparable to every other weight — it is the rupee change as that feature moves from its minimum to its maximum. That is why `fuel_consumption_litres` at +2938 is instantly identifiable as the dominant driver.

Part A accuracy (on the training data)

R^2	MSE	RMSE	MAE
0.992841	4284.6703	₹65.4574	₹48.2964

5.4 Part B (warm-up) — one feature, by hand

Before doing six features at once, the classic $y = mx + c$ case, using `distance_km` alone. The three steps of gradient descent:

1. Loss function — Mean Squared Error, i.e. how wrong the current line is:

$$L(m, c) = (1/n) \sum (Y_i - (mX_i + c))^2$$

2. Gradients — the direction that *increases* the loss:

$$\partial L / \partial m = (2/n) \sum X_i \cdot \text{error}_i \quad \partial L / \partial c = (2/n) \sum \text{error}_i \quad \text{where error} = \hat{Y} - Y$$

3. Update rule — step in the *opposite* direction; that is the “descent”:

$$m \leftarrow m - lr \cdot \partial L / \partial m \quad c \leftarrow c - lr \cdot \partial L / \partial c$$

Result of the warm-up

Parameter	sklearn	Gradient descent	Abs. difference	% difference
m (slope)	2695.202781	2695.199903	0.002878	0.000107%
c (intercept)	1109.318269	1109.319377	0.001108	0.000100%

Learning rate 0.1, budget 20,000 epochs, **converged at epoch 1,339** once the loss changed by less than $1e-8$. R^2 using distance alone: 0.6914 — which is exactly why the multi-feature model is needed.

5.5 Part B — all six features

Extending `m`, `c` to `w`, `b`: one weight per feature plus a single bias.

$$\hat{Y} = Xw + b \quad L(w, b) = (1/n) \sum (\hat{Y}_i - Y_i)^2$$

$$\partial L / \partial w_j = (2/n) \sum_i X_{ij} \cdot \text{error}_i \quad \partial L / \partial b = (2/n) \sum_i \text{error}_i$$

All six gradients are computed in a **single matrix operation**, `X.T @ error`, rather than one Python variable per feature. This is the vectorised form — it is both far faster and closer to the mathematics.

```
def gradient_descent(X, Y, lr=0.2, epochs=50000, tol=1e-8):
    n, k = X.shape
    w = np.zeros(k)          # initial guess: all weights zero
    b = 0.0
    loss_history = []
    for i in range(epochs):
        Y_pred = X.dot(w) + b          # current prediction
        error = Y_pred - Y             # predicted - actual
        loss = (1 / n) * np.sum(error ** 2)  # MSE
        loss_history.append(loss)

        dw = (2 / n) * X.T.dot(error)    # gradient for every weight at once
        db = (2 / n) * np.sum(error)      # gradient for the bias

        w = w - lr * dw                 # update
        b = b - lr * db

        if i > 0 and abs(loss_history[-2] - loss_history[-1]) < tol:
            break                       # loss has gone flat: converged
    return w, b, loss_history, i + 1
```

The run

Learning rate **0.2**, budget 50,000 epochs, **converged at epoch 14,317**.

Epoch	MSE loss
0	4,989,244.12
10	180,559.59
100	11,087.46
500	7,131.79
1,000	5,670.92
5,000	4,289.12
10,000	4,284.67
14,316	4,284.6703 ← final

The shape of that curve is the whole story: a near-vertical drop in the first ~100 epochs, then 14,000 epochs spent squeezing out the last few decimal places. Three views of it were plotted — the raw curve, a zoom on the first 500 epochs, and a log-scale version showing the long flat tail.

5.6 The comparison — do they match?

Parameter	sklearn	Gradient descent	Abs. diff
distance_km	225.038176	225.073171	0.034995
mileage	-79.410245	-79.434153	0.023908
fuel_price	636.323238	636.32	< 0.01
toll_cost	547.348156	547.35	< 0.01
parking_cost	165.487549	165.49	< 0.01
fuel_consumption_litres	2938.494104	2938.45	0.042477
intercept	301.323450	301.339700	0.016250
Largest absolute difference on any parameter			0.042477
Largest percentage difference			0.030107%

And the predictions?

Metric	sklearn	Gradient descent
R ²	0.992841	0.992841
RMSE	65.4574	65.4574
MAE	48.2964	48.2965
Largest gap between the two models' predictions across all 1,129 trips: ₹0.011354		

Why they must match — the answer to expect to give

The MSE loss for linear regression is a **convex** function — a bowl with exactly one minimum and no local minima to get stuck in. sklearn jumps straight to the bottom by solving the normal equations algebraically. Gradient descent walks down to the same bottom one step at a time. Given a sensible learning rate and enough epochs, **both must land in the same place, because there is nowhere else to land.**

The largest disagreement on any parameter is **4 paise**, against coefficients in the hundreds and thousands.

5.7 Where a difference *would* come from

That the two agree is not automatic. Two experiments were run to show what goes wrong otherwise.

Experiment 1 — learning rate, fixed budget of 3,000 epochs

Learning rate	Max weight error vs sklearn	Diagnosis
0.001	≈ 2025	Far too small — loss still falling steeply when the loop ended
0.01	≈ 1168	Still undertrained
0.05	≈ 719	Closer
0.20	≈ 143	Good — the value used
> ≈ 0.44	diverges	Updates overshoot the minimum; loss oscillates then goes to <code>inf / NaN</code>

Experiment 2 — epoch budget, learning rate fixed at 0.2

Epochs	100	1,000	10,000	50,000
Max weight error	≈ 1260	≈ 600	0.94	≈ 0

The four failure modes, and how each shows up

Cause	Symptom	Fix used here
Learning rate too small	Loss still falling when the loop ends; weights stuck near their starting values	<code>lr = 0.2</code>
Too few epochs	Same undertrained result even at a good learning rate	Budget 50,000, early-stopped at 14,317
Learning rate too large	Updates overshoot; loss oscillates and blows up to <code>inf / NaN</code>	Kept below the measured ≈ 0.44 ceiling
Unscaled features	Features on very different ranges make the loss surface a narrow valley; one learning rate cannot suit all weights	MinMax scaling in Week 2, Step 7

5.8 Using the model on a new trip

A new trip must pass through the *same* Week 2 scaler before either model can see it — a point worth stressing, because fitting a fresh scaler on one row would be meaningless.

Input (raw units)	Value
distance_km	250.0
mileage	16.0
fuel_price	95.0
toll_cost	260.0
parking_cost	80.0
fuel_consumption_litres	15.6

Model	Prediction
sklearn	₹1,990.78
Gradient descent from scratch	₹1,990.78
Difference	₹0.002191

5.9 Week 3 conclusion

Do the sklearn parameters and the gradient-descent parameters match? Yes — to within 0.03%, for both the single-feature warm-up and the six-feature model. Predictions never differ by more than **₹0.011** across all 1,129 trips, R^2 is identical to six decimal places, and RMSE and MAE agree to three.

Week 3 ended with $R^2 = 0.992841$ and a note that the remaining 0.7% looked “structural”. That number was suspiciously high — which is what sent Week 4 looking for the reason.

Proving the dataset is synthetic

6.1 The question that started it

Week 3 ended at $R^2 = 0.9928$. That is a suspiciously good score for a dataset with weather, traffic and human behaviour in it. So Week 4 asked a different question — **not “how well can we predict the target” but “what actually generated it?”**

The method is deliberately simple: **subtract the cost components we already know about and study what is left**. If the residual is structured rather than random, the data came from a formula.

```
raw['fuel_cost'] = raw.fuel_consumption_litres * raw.fuel_price
raw['residual'] = raw.total_trip_cost - raw.fuel_cost - raw.toll_cost - raw.parking_cost
```

6.2 The residual is not noise

count	mean	std	min	25%	50%	75%	max
1140	166.16	67.29	48.50	111.59	159.43	206.47	448.25

Two facts jump out. The residual is **never negative and never zero** — so something is systematically missing, not randomly scattered. And it correlates **+0.8462 with distance**. Something proportional to distance is unaccounted for.

6.3 Finding which column controls it

Dividing the residual by distance gives a per-kilometre rate. Grouping that rate by each categorical column shows which one governs it:

Grouped by	Mean residual per km, by category	Verdict
vehicle_type	Hatchback 0.5694 · Sedan 0.6943 · SUV 0.8366 (sd 0.077–0.108)	This is it — a clean 47% spread
fuel_type	CNG 0.6958 · Diesel 0.6890 · Petrol 0.7006	No effect — all within noise
traffic_level	High 0.6862 · Low 0.7008 · Medium 0.6968	No effect
weather_condition	Clear 0.6917 · Cloudy 0.7032 · Fog 0.7147 · Rain 0.6923	No effect

Only `vehicle_type` splits it, and it splits it cleanly. That residual is a **per-kilometre maintenance / wear-and-tear charge that depends on the vehicle**.

6.4 The recovered formula

```
total_trip_cost = fuel_consumption_litres x fuel_price
                  + toll_cost
                  + parking_cost
                  + distance_km x maintenance_rate(vehicle_type)

maintenance_rate (Rs/km):  Hatchback 0.5694  Sedan 0.6943  SUV 0.8366

fuel_consumption_litres = (distance_km / mileage) x traffic_multiplier
traffic_multiplier:      Low 0.9445  Medium 1.0446  High 1.1653
estimated_time_min/km:  Low 0.919   Medium 1.059   High 1.263
toll = distance x N(1.301, 0.360) Rs/km
parking is drawn uniformly from {0, 40, 60, 80, 120, 150}
weather_condition has NO effect on anything
```

How well does the recovered formula reproduce the data?

R ² of reconstruction	Mean absolute error	Max absolute error
0.999162	₹18.44	₹66.85

The traffic multipliers were confirmed independently by taking the ratio of actual litres to ideal litres (`distance / mileage`) and grouping by traffic level:

Traffic level	mean ratio	min	max
Low	0.9445	0.9118	0.9770
Medium	1.0446	1.0089	1.0817
High	1.1653	1.1256	1.2063

Grouped by weather instead: Clear 1.0513, Cloudy 1.0434, Fog 1.0408, Rain 1.0478 — flat, confirming weather does nothing.

What this result means, and why it is the intellectual centre of the project

R² = 0.999162 from arithmetic alone. No model was fitted — the formula was recovered by inspection and then evaluated. Two consequences follow:

- **Week 3's R² = 0.9928 was measuring formula recovery, not predictive skill.** The model was rediscovering an equation that generates the data exactly, which is why the score was so high.
- It also unlocks Week 5: because the generating physics is known, a much larger dataset can be generated over *real* geography that is **physically consistent** with the original.

This is also the reason the project reports two accuracy figures. If the target is a formula and the model is handed three of its four terms, a near-perfect score is arithmetic, not intelligence — the point Weeks 9 and 10 pursue to its conclusion.

Scaling from 9 routes to 534 cities

7.1 Why this was necessary

The original dataset covers only **9 fixed city pairs**, 110–460 km. A model trained on it cannot serve a user who wants Kochi to Nagpur — it has never seen a route it was not given. To build something usable, the dataset had to cover arbitrary Indian routes.

7.2 How the wide dataset was built

Step	Detail
City source	GeoNames <code>cities15000</code> → 3,739 Indian cities with real latitude/longitude (CC BY 4.0)
Filter	Population ≥ 100,000 → 537 cities in domain
Route sampling	5,000 distinct pairs, sampled with a short-trip preference (decay 350 km) so the set resembles real road trips rather than random long-haul pairs
Trips per route	5 → 25,000 trips , covering 534 distinct cities
Distance band	50–1,500 km
Straight-line distance	Haversine on the real coordinates
Road distance	$\text{haversine} \times N(1.2355, 0.082)$ — the winding factor measured on the project's own 9 routes
Costs	Generated with the Week 4 formula, so they are physically consistent with the original data

Resulting distance distribution across the 25,000 trips:

count	mean	std	min	25%	50%	75%	max
25,000	485.0	319.4	50.0	236.0	412.5	667.5	1,500.0

7.3 What is real and what is modelled — stated openly

Status	Which parts
Real	City names, states, latitude and longitude (GeoNames, 3,739 cities). Straight-line distances computed by haversine on those real coordinates.
Calibrated	Road distance = haversine × 1.2355, measured on the project's own 9 routes and cross-checked against OSRM (agreed within 5–9%). Maintenance rate, traffic multipliers, toll rate and mileage ranges — all measured from the original CSV in Week 4.
Partly real	Fuel prices — 6 states verified from public reporting, 29 estimated. <code>data/fuel_prices.csv</code> records which is which per row.
Generated	Every trip record: vehicle, fuel, mileage, departure hour, month, traffic, weather, and all cost columns.

The one deliberate departure from the original data — disclose this before being asked

In the original CSV, `traffic_level` is **statistically independent** of `departure_hour`: $P(\text{High})$ wanders between 0.13 and 0.35 with no rush-hour pattern. That makes traffic unpredictable, and any traffic classifier trained on it can do no better than the majority baseline.

The generator therefore gives traffic a **real rush-hour profile** — morning 8–11 and evening 17–21 weighted toward High, night weighted toward Low:

Window	P(Low)	P(Medium)	P(High)
Peak (8–11, 17–21)	0.10	0.35	0.55
Shoulder (6–7, 12–16, 22)	0.30	0.50	0.20
Night / early morning	0.62	0.31	0.07

Weather is also made monsoon-aware (June–September weighted to Rain, December–January to Fog).

This is more realistic AND it makes traffic classification a genuine task rather than decoration. It is a modelling choice, and it means the Week 7 traffic accuracy measures that design decision rather than Indian roads. Part 11 shows the two side by side.

One engineered feature beats two ensembles

8.1 The testable claim

Week 3 named the linear model's structural blind spot:

The real fuel bill is the *product* `litres × price`, and a linear model can only **add** its features together, never multiply them.

That is a claim that can be tested. If the missing product is really the bottleneck, then **supplying it as a feature should beat throwing a nonlinear model at the problem** — because a tree ensemble has to *approximate* a product it can never represent exactly, whereas we can simply hand it over.

8.2 The experiment

Four models, same 25,000 rows, same 80/20 split, same six base features (`distance_km`, `mileage`, `fuel_price`, `toll_cost`, `parking_cost`, `fuel_consumption_litres`):

Model	Test R ²	Test MAE
Linear (no interaction)	0.994666	₹143.85
RandomForest, 300 trees	0.998543	₹69.97
GradientBoosting	0.998560	₹73.50
Linear + <code>litres × price</code>	0.999614	₹42.60

The prediction holds. One engineered column takes MAE from ₹143.85 to ₹42.60 — a **3.4× improvement** — and beats both ensembles, which had access to exactly the same information but had to learn the product implicitly.

8.3 The second missing feature

Week 4 found the per-km maintenance rate splits by vehicle: Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 — a 47% spread. None of the models above could see it, because `vehicle_type` is categorical and had never been encoded into the regression. That meant they returned the **same cost for an SUV and a hatchback on an identical route**, which is both wrong and obviously wrong to any user.

One-hot encoding it (Hatchback as baseline) gives the model that ships from Week 6 onward:

Model	Test R ²	Test MAE
Linear + interaction + vehicle one-hots	0.999707	₹39.66

Learned per-vehicle offsets, relative to Hatchback: **SUV +₹129.63, Sedan +₹64.43** — the correct ordering, recovered from data.

Two things worth drawing out

- **Domain understanding beat model complexity.** The win came from knowing what the number *means*, not from a bigger hypothesis class.
- **The model stays linear.** Everything from Week 3 — the closed-form solution, the from-scratch gradient descent, the interpretable coefficients — still applies. A Random Forest would have cost all of that *and* performed worse.

Note for Part 12: this conclusion holds when distance and litres are *given*. Week 10 re-runs the same experiment when the model must infer distance from coordinates, and the result reverses. Knowing the boundary of your own finding is worth more than the finding.

Classification — and which target is real

The task asked for classification where it adds value. Two targets were available, and they are **not of equal quality**. Saying which is which is part of the work.

9.1 Target 1 — `traffic_level` (the genuinely hard one)

First, why this was *impossible* on the original dataset. Cross-tabulating $P(\text{traffic} \mid \text{hour})$ across the 24 hours of the original CSV:

Finding	Value
P(High) across hours	wanders between 0.13 and 0.35 with no rush-hour structure
Majority-class baseline	0.5088
Conclusion	Traffic is independent of departure hour → no classifier can beat the baseline

On the wide dataset, which has the injected rush-hour profile, the task becomes learnable.

Feature encoding — and why hour is not fed as 0–23

Departure hour is **cyclical**: 23:00 and 00:00 are one hour apart, not twenty-three. Feeding the raw integer would make the model treat them as maximally distant. So hour and month go in as sine/cosine pairs:

```
hour_sin = sin(2*pi*hour/24)    month_sin = sin(2*pi*month/12)
hour_cos = cos(2*pi*hour/24)    month_cos = cos(2*pi*month/12)
is_peak  = 1 if (8 <= hour <= 11) or (17 <= hour <= 21) else 0
```

`is_peak` is included because sin/cos alone cannot express “high between 8–11 *and again* between 17–21” — that is a two-humped shape a single sinusoid cannot make, so a linear model is handed it directly.

Result (Week 7, LogisticRegression)

Accuracy	Majority baseline	Lift
53.9%	38.8%	+15.1 points

Modest, and correct to be modest. Departure hour genuinely only *partly* determines traffic. A classifier claiming 95% here would be evidence of leakage, not skill.

9.2 Target 2 — `cost_band` (useful, but high by construction)

The second target is a **quartile split of ₹/km** into Excellent / Good / Average / Poor. Note: cost *per kilometre*, not absolute cost — absolute cost is essentially determined by distance, so classifying it would tell a user nothing. Efficiency isolates the effect of vehicle, mileage, fuel and traffic.

Accuracy	Baseline (4 balanced classes)
85.8%	25%

85.8% is high by construction, not because the problem is hard. `cost_band` is a discretisation of a quantity the Week 6 regressor already predicts to within ₹42. It earns its place because the interface needs the label, not because it demonstrates difficult learning. **The traffic classifier is the one that shows real modelling.** Volunteering this distinction is stronger than being caught by it.

9.3 How the classifier earns its place in the product

A classifier that outputs one word is decoration. This one drives a feature: after an estimate, the site charts expected cost across **all 24 departure hours**.

The bars are the classifier's **probabilities weighted against the cost at each traffic level** — *not* the cost at the single most likely level. Taking the argmax would collapse 24 hours onto 3 values and discard everything the model knows about its own confidence: an hour that is 55% High reads very differently from one that is 95% High.

$$E[\text{cost} \mid \text{hour}] = \sum_{\text{levels}} P(\text{level} \mid \text{hour}) \times \text{cost}(\text{level})$$

The chart also takes a **departure window** — the hours you could realistically leave in, wrapping past midnight for overnight drives. Hours outside it dim but stay visible as context, and the recommendation is the best hour *you can actually use*: “leave at 1am” is not advice most people can act on. On Jaipur → Agra the full day spreads **₹177**; restricted to a 6–11am departure it is still **₹152**.

Why gradient descent stopped converging

10.1 The symptom

The Week 3 from-scratch gradient descent was re-run on the 25,000-row dataset with the Week 6 feature set. It behaved differently:

Measure	Value
R ² , sklearn vs gradient descent	0.999707 vs 0.999701
Largest gap between the two models' predictions	₹82.81
Largest gap between the two models' weights	682.72
Epochs	60,000 – did not early-stop

Predictions agree; weights do not converge. On the original 1,129 rows the largest weight gap was 0.042. Here it is 682.

10.2 The cause, measured

The Week 6 interaction term is `fuel_bill = fuel_consumption_litres × fuel_price`. That makes `fuel_bill` almost a deterministic function of `fuel_consumption_litres`:

Measure	Value
corr(litres, fuel_bill)	0.989
Condition number of the Hessian, κ(H)	≈ 16,000

Two near-collinear columns make the loss surface a **long, narrow valley** rather than a round bowl. Many different weight combinations fit almost equally well. Gradient descent descends into the valley quickly and then crawls along its nearly-flat floor; sklearn's normal equations jump straight to one specific point. **Both are correct — they simply pick different points in the same valley**, which is why the predictions match while the weights do not.

10.3 Ridge fixes the conditioning, and costs too much

Ridge regression adds an L2 penalty, which lifts the flat direction of the valley:

$$\text{minimise } ||Xw - y||^2 + \alpha ||w||^2 \rightarrow w = (X^T X + \alpha I)^{-1} X^T y$$

α	cond(H)	GD weight gap @ 20k epochs	Test MAE
0 (OLS)	16,155	4090.89	₹39.66
10	2,284	0.14	₹109.41
100	263	0.00	₹255.42

Ridge does exactly what the theory promises — κ falls $\sim 50\times$ and gradient descent converges essentially exactly — and it costs **six times the error** to do it.

Question	Answer
Does regularisation fix the conditioning?	Yes — κ falls $\sim 50\times$
Does it make gradient descent converge?	Yes — gap 4113 $\rightarrow$ 0.00
Should we use it for this model?	No

The conclusion — a strong viva answer

We do not actually *need* gradient descent. The normal equations solve this model exactly, in one step, and are **indifferent to the condition number**. Gradient descent was an *exercise* in understanding what `fit()` does — and the exercise taught something more useful than “the two methods agree”:

An ill-conditioned problem is not a broken model. It is a problem where iterative solvers struggle and direct solvers do not. Reaching for Ridge here would trade real accuracy to fix a difficulty that only exists because of the solver we chose.

So the model that ships is ordinary least squares with the Week 6 interaction term, solved in closed form. Week 9 then confirms this mechanically: a `GridSearchCV` over α drives it to the bottom of the grid without being told any of the above.

Also measured: the divergence threshold for the learning rate on this data is $\text{lr} \approx 0.45$ (0.5 blows up to `inf`), closely matching the ≈ 0.44 ceiling found on the original CSV. Training uses 0.4. All of this is surfaced in the application through the `/api/loss-curve` endpoint rather than hidden.

Task 5 — evaluation, validation and tuning

Weeks 1–8 built models. Week 9 measures them properly, applying the same six steps to every model in the project. This is the Task 5 checklist, answered in full.

Step	Regression	Classification
1. Metrics	RSS, RMSE, R^2	Accuracy, Precision, Recall, F1
2. Fit diagnosis	train score vs test score	same
3. Validation	5-fold cross-validation + bootstrap	5-fold stratified
4. Comparison	one table, best score <i>and</i> stable CV	same
5. Tuning	GridSearchCV / RandomizedSearchCV	same
6. Advanced models	RandomForest (bagging), AdaBoost, GradientBoosting	same

11.1 How everything was measured

A comparison only means something if every row of it was measured the same way, so two things are fixed once and reused everywhere.

One split for everything. `train_test_split(test_size=0.2, random_state=42)` gives **20,000 training rows and 5,000 test rows**, and every table below is scored on those same 5,000 rows.

Every model wrapped in `Pipeline([MinMaxScaler, model])`. Linear and logistic models need the scaling and trees do not care, so the reason to do it for all of them is cross-validation correctness: inside a Pipeline the scaler is re-fitted on each training fold, so nothing from the validation fold reaches it. Scaling the whole matrix once *before* splitting — which this project originally did — lets test-set minima and maxima leak into training. The effect is small for MinMaxScaler on this data, but it is exactly the mistake cross-validation exists to catch.

11.2 Step 1 — the metrics, and what each is for

$$RSS = \sum_i (y_i - \hat{y}_i)^2 \cdot RMSE = \sqrt{RSS / n} \cdot R^2 = 1 - RSS / \sum_i (y_i - \bar{y})^2$$

Metric	What it is, and when to quote it
RSS	Residual sum of squares — the raw quantity least squares minimises. It is a <i>sum</i> , so it grows with the number of rows: comparable between models on the same split, meaningless across splits of different sizes.
RMSE	RSS per row, square-rooted, so it reads in rupees. Squaring inside means one ₹2,000 miss counts as much as a hundred ₹200 misses — quote it when large errors are what hurt.
MAE	Mean absolute error — average error in rupees, no squaring. This is the number to tell a user.
R^2	Fraction of variance explained. Its weakness is exactly what this project ran into: when the target is nearly deterministic, R^2 sits so close to 1 that differences which matter in rupees become invisible.

$$Precision = TP / (TP + FP) \cdot Recall = TP / (TP + FN) \cdot F_1 = 2PR / (P + R)$$

Precision, recall and F1 are **macro-averaged** (every class weighted equally) rather than weighted, because weighted averaging can hide a class the model never predicts at all. Part 11.10 shows exactly that failure on the original dataset.

11.3 Steps 1 and 4 — the regression comparison

Target `total_trip_cost`. Feature set: the model this project already ships (Week 6). Every row sees exactly those columns and that split, so the table compares **models**, not feature sets. The first row is the Week 6 ablation (interaction term removed), included because it is the honest baseline the headline beats.

Model	RSS	RMSE (₹)	R ²	MAE (₹)
Linear, NO interaction	2.606e+08	228.31	0.994795	139.75
LinearRegression	1.467e+07	54.16	0.999707	39.66
Ridge (alpha=1)	2.843e+07	75.40	0.999432	52.63
DecisionTree	6.948e+07	117.88	0.998612	79.52
RandomForest (300)	3.762e+07	86.75	0.999249	52.92
AdaBoost	7.944e+08	398.59	0.984135	313.86
GradientBoosting	3.871e+07	87.99	0.999227	56.76

Read this table from the RMSE column, not R². Every model except AdaBoost scores $R^2 > 0.998$, which invites the conclusion that the choice does not matter. In rupees it clearly does: ₹54 for the linear model against ₹399 for AdaBoost — the worst model is about 7 times worse than the best while looking like a rounding difference in R^2 .

The largest single effect in the table is not a model at all. Removing one engineered column (`fuel_bill = litres × price`) moves RMSE from ₹54.16 to ₹228.31 — a bigger gap than between the best and worst *model*. That is Week 6's argument restated: an additive model cannot express a product, and no amount of model complexity substitutes for handing it over.

11.4 Step 2 — overfitting and underfitting

Verdicts need **stated thresholds** to be reproducible. A verdict that depends on an unwritten judgement call is not a result.

Condition	Verdict
train R^2 – test R^2 > 0.05	Overfitting
both R^2 < 0.50	Underfitting
accuracy less than 3 points above the majority baseline	Underfitting (classification)
otherwise	Good fit

Model	Train R^2	Test R^2	Gap	Verdict
Linear, NO interaction	0.994853	0.994795	+0.000058	Good fit
LinearRegression	0.999715	0.999707	+0.000008	Good fit
Ridge (alpha=1)	0.999457	0.999432	+0.000025	Good fit
DecisionTree	1.000000	0.998612	+0.001388	Good fit
RandomForest (300)	0.999906	0.999249	+0.000658	Good fit
AdaBoost	0.984114	0.984135	-0.000020	Good fit
GradientBoosting	0.999462	0.999227	+0.000236	Good fit

Every model says “Good fit” — and that is a fact about the data

`DecisionTree` reaches **train $R^2 = 1.000000$** . A tree grown to purity has memorised all 20,000 training rows — the textbook overfitting signature. It nevertheless holds test $R^2 = 0.9986$, because the target here is a deterministic formula: memorising the training set and learning the rule produce nearly the same predictions when there is almost no noise to memorise.

So **on this target the train/test comparison is close to uninformative**. It becomes informative the moment there is real noise — which happens in two places later: the `cost_band` classifier below, and the toll sub-model in Part 12, where the same trees show gaps of 0.16 to 0.37 and are correctly flagged.

11.5 Step 3 — 5-fold cross-validation, and why the spread is the point

A single train/test split gives one number, and that number depends on which rows happened to land in the test set. K-fold splits the *training* data into k parts, trains on k-1 and validates on the remaining one, k times. Two things come out: a mean (a better estimate than one split) and a **standard deviation across folds** — which is what says whether the model is stable or merely lucky.

Model	CV mean R^2	CV std	Fold range
Linear, NO interaction	0.994846	0.000125	0.000322
LinearRegression	0.999714	0.000010	0.000026
Ridge (alpha=1)	0.999387	0.000030	0.000078
DecisionTree	0.998292	0.000092	0.000244
RandomForest (300)	0.999266	0.000080	0.000233
AdaBoost	0.983787	0.000447	0.001347
GradientBoosting	0.999267	0.000083	0.000239

Every spread is tiny — CV standard deviations of $1e-5$ to $4e-4$. With 20,000 training rows and a near-deterministic target that is the expected result, and worth stating plainly: **this cross-validation confirms stability, it does not discriminate between the models**. Part 11.8 repeats it on 1,140 rows, where it does.

Bootstrap — the checklist's alternative, run as well

K-fold asks *how does the score vary across disjoint held-out folds*. Bootstrap asks *how would the score vary if I had drawn a different training sample of the same size* — resampling the training rows with replacement, refitting, and scoring each fit on the fixed test split.

Quantity	Value
Model	LinearRegression
Resamples	200
Mean R^2	0.999707
Standard deviation	0.00000062
95% interval	[0.999705, 0.999708]
Interval width	0.00000252

The interval is about $3e-6$ wide: the fit does not depend in any meaningful way on which particular rows were drawn. The model is stable, not lucky.

11.6 Step 4 — choosing between models that are statistically tied

Task 5 asks for “best score **and** stable cross-validation result”, which has to be made precise. `pick_best` applies three filters in order.

Filter	Rule	Why
1. Test-error band	keep models within 2% of the best RMSE (or 0.5 accuracy points)	Closeness judged on <i>error</i> , never on R^2 . See the box below.
2. One-standard-error rule	of those, keep every model whose CV mean is within one standard error ($sd/\sqrt{k}$) of the best CV mean	Standard practice from Hastie, Tibshirani & Friedman, <i>The Elements of Statistical Learning</i> §7.10: a difference smaller than the noise in the CV estimate itself is not a real difference.
3. Simplicity	of the survivors, take the simplest	Not only Occam's razor — a linear model keeps the Week 3 gradient-descent-from-scratch code applicable, which no ensemble does.

A bug this project found in its own selection code

A first version of `pick_best` treated two models as tied when their R^2 was within 0.5% *relative*. Near $R^2 = 0.99$ that band is wide enough to swallow a model with 15% more RMSE — and it duly preferred a Ridge fit at 53.7 km RMSE over a random forest at 46.6 km, because the Ridge fit had a marginally steadier CV score. Judging closeness on the error instead fixed it. This is a good example to volunteer: it shows the evaluation code itself was tested, not trusted.

Result: `LinearRegression` for cost regression, and `GradientBoosting` for `traffic_level`, `LogisticRegression` for `cost_band`.

11.7 Steps 1–4 for classification

Target: `traffic_level`

Model	Accuracy	Precision	Recall	F1	Train acc	Gap	Verdict
LogisticRegression	0.5456	0.5393	0.5634	0.5373	0.5277	-0.0179	Good fit
DecisionTree	0.5604	0.5639	0.5696	0.5641	0.5531	-0.0073	Good fit
RandomForest (300)	0.5604	0.5639	0.5696	0.5641	0.5531	-0.0073	Good fit
AdaBoost	0.5692	0.5718	0.5795	0.5718	0.5504	-0.0188	Good fit
GradientBoosting	0.5696	0.5729	0.5797	0.5724	0.5517	-0.0179	Good fit

Majority-class baseline **0.3766** → best model `GradientBoosting` gains **+19.3 points**, with CV 0.5517 ± 0.0045 .

Confusion matrix — `GradientBoosting` (rows = actual, columns = predicted)

actual ↓ / predicted →	High	Low	Medium
High	1,092	95	334
Low	173	927	496
Medium	641	413	829

Target: `cost_band`

Model	Accuracy	Precision	Recall	F1	Train acc	Gap	Verdict
LogisticRegression	0.8562	0.8580	0.8574	0.8577	0.8563	+0.0001	Good fit
DecisionTree	0.7918	0.7931	0.7936	0.7934	1.0000	+0.2082	Overfitting
RandomForest (300)	0.8354	0.8385	0.8367	0.8375	1.0000	+0.1646	Overfitting
AdaBoost	0.6340	0.6655	0.6328	0.6383	0.6297	-0.0043	Good fit
GradientBoosting	0.8194	0.8237	0.8210	0.8222	0.8470	+0.0276	Good fit

Majority-class baseline **0.2568** → best model `LogisticRegression` gains **+59.9 points**, with CV 0.8523 ± 0.0039 .

Confusion matrix — `LogisticRegression` (rows = actual, columns = predicted)

actual ↓ / predicted →	Average	Excellent	Good	Poor
Average	1,051	9	131	93
Excellent	0	1,074	130	0
Good	137	113	1,033	0
Poor	104	0	2	1,123

The two targets are not of equal quality, and saying so is part of the answer

`traffic_level` is the genuinely non-trivial one: departure hour only partly determines traffic, so an honest result looks modest, and a classifier claiming 95% here would be evidence of leakage. Note also *how* it fails — the confusion matrix concentrates errors between *adjacent* levels (Low with Medium, Medium with High) and is best at the extremes. That is the right failure mode for an ordered target.

`cost_band` is a quartile split of ₹/km that the regressor already predicts well, so its high accuracy is **by construction**. It earns its place because the interface needs the label, not because it is hard. But it is where the overfitting diagnosis finally bites: the two tree models hit train accuracy 1.0000 with test well below, and `LogisticRegression` — with almost no capacity to memorise — wins outright.

11.8 Step 5 — hyperparameter tuning

Why not grid-search the winning model? The best regressor is `LinearRegression`, which has no hyperparameter worth tuning — a grid search over it is theatre. Ridge is the same model plus exactly one knob, so searching α is a real search whose answer means something: **Week 8 rejected Ridge by hand**, arguing from the Hessian condition number. If that argument was right, the search should drive α to the bottom of the grid unaided.

(a) GridSearchCV — Ridge α

α	CV R^2	CV sd
0.0001	0.999714	0.000015
0.001	0.999714	0.000015
0.01	0.999714	0.000015
0.1	0.999704	0.000016
1	0.999387	0.000019
10	0.997173	0.000095
100	0.983673	0.000500
1000	0.835241	0.002508

	Test R^2	RMSE (₹)
sklearn default $\alpha = 1.0$	0.999432	75.40
after the search ($\alpha = 0.001$)	0.999707	54.16

The CV score is **flat for every $\alpha \leq 0.01$** and falls monotonically above it, collapsing to R^2 0.8352 by $\alpha = 1000$. Across the flat region the penalty is too small to do anything, so Ridge there is ordinary least squares; the search has no reason to prefer any point on the plateau and returns one of them. Every α large enough to actually regularise scores strictly worse.

This reproduces Week 8's hand-argument mechanically. A search whose answer is “do not regularise” has not failed — it has confirmed the model was already correctly specified. Note the “improved” verdict below is

against sklearn's *default* $\alpha = 1.0$, and the improvement consists of turning regularisation off.

(b) RandomizedSearchCV — RandomForestRegressor

Search space, 12 random draws $\times$ 5 folds:

```
{
  "n_estimators": [
    100,
    200,
    300,
    500
  ],
  "max_depth": [
    null,
    8,
    14,
    20
  ],
  "min_samples_leaf": [
    1,
    2,
    5,
    10
  ],
  "max_features": [
    1.0,
    0.7,
    0.5
  ]
}
```

	Test R^2	MAE (₹)
300 trees at defaults	0.999249	52.92
tuned	0.999187	53.42

Best parameters found: `{'n_estimators': 300, 'min_samples_leaf': 1, 'max_features': 0.7, 'max_depth': None}`, best CV R^2 0.999223. Outcome: **NO IMPROVEMENT** (+0.50 ₹ MAE).

(c) GridSearchCV — the traffic classifier

```
{
  "n_estimators": [
    100,
    300
  ],
  "max_depth": [
    null,
    4,
    8
  ],
  "min_samples_leaf": [
    1,
    10,
    50
  ]
}
```

	Accuracy	F1 (macro)
300 trees at defaults	0.5604	0.5641
tuned	0.5696	0.5724

Best parameters: `{'max_depth': 4, 'min_samples_leaf': 1, 'n_estimators': 100}`, best CV accuracy 0.5517, majority baseline 0.3766. **Improved by +0.92 accuracy points.**

This is the one search with room to work. An unrestricted forest memorises the training rows; capping `max_depth` at 4 stops it, and the *test* score goes up while the training score comes down. That is the same finding as the overfitting table, reached from the other direction.

“Confirm score improved” — 2 of 3 did

Search	Outcome	Why
Ridge α	improved	...by turning regularisation off . The real finding is that this problem wants none.
RandomForest	no improvement	Nothing left to find — a linear model already fits the formula to R^2 0.9997.
Traffic classifier	improved	The one target with genuine noise, so constraining model capacity buys something.

The answer to give if asked whether tuning worked

Two of the three searches improved the score, one did not, and **the pattern is the lesson**: hyperparameter tuning pays where the data is *noisy* and the model can overfit it. Where the target is a formula and the features already span it, tuning has nothing to do — and a search that reports no improvement is evidence the model was specified correctly, not evidence the search was wasted.

11.9 Step 6 — the advanced models, in one place

RandomForest (bagging), AdaBoost and GradientBoosting appear in every table rather than in a section of their own, because the point of including them is comparison.

Model	Test R^2	RMSE (₹)	MAE (₹)	Verdict
RandomForest (300)	0.999249	86.75	52.92	Good fit
AdaBoost	0.984135	398.59	313.86	Good fit
GradientBoosting	0.999227	87.99	56.76	Good fit
<i>LinearRegression — for comparison</i>	0.999707	54.16	39.66	Good fit

- **Bagging (RandomForest)** — trains many trees on bootstrap samples and averages them, which cuts variance. Strongest ensemble here, and still beaten by a linear model on the cost target. It wins where the relationship is genuinely non-linear: the road-distance sub-model and the end-to-end model, both of which must learn geography.
- **Boosting (AdaBoost)** — fits shallow stumps sequentially, re-weighting the rows it got wrong. Consistently weakest here, and worth knowing *why*: on a smooth additive target it spends its capacity chasing the noisiest rows instead of representing the smooth part.
- **GradientBoosting** — fits each new tree to the residuals of the previous ones. Close behind RandomForest throughout, and it wins the traffic classification outright, where the signal is weak.

None of the three beats a correctly specified linear model on the cost target. That is the project's recurring result, and it survived every test in this part.

11.10 The same checklist on the original 1,140-row dataset

The assignment was handed out with `road_trip_data.csv` (1,140 rows), so the tables are repeated on it. It makes two points the wide dataset cannot.

Point 1 — small data is measurably less stable

Model	Test R^2	RMSE (₹)	CV sd (1,140 rows)	CV sd (25,000 rows)	Ratio
LinearRegression	0.998902	25.32	0.000037	0.000010	3.6×
Ridge (alpha=1)	0.997435	38.69	0.000371	0.000030	12.5×
DecisionTree	0.987789	84.42	0.001811	0.000092	19.7×
RandomForest (300)	0.992900	64.37	0.000907	0.000080	11.4×
AdaBoost	0.973382	124.64	0.002752	0.000447	6.2×
GradientBoosting	0.996449	45.53	0.000824	0.000083	10.0×

Best on the original data: **LinearRegression**. The cross-validation standard deviation is **4 to 20 times wider** for the same models and the same code. This is the clearest argument in the project for why step 3 is on the checklist at all — with one split you would never see it.

Point 2 — on the original data, traffic classification is impossible

Model	Accuracy	Precision	Recall	F1	Verdict
LogisticRegression	0.5088	0.1696	0.3333	0.2248	Underfitting
DecisionTree	0.5088	0.1696	0.3333	0.2248	Underfitting
RandomForest (300)	0.5088	0.1696	0.3333	0.2248	Underfitting
AdaBoost	0.5088	0.1696	0.3333	0.2248	Underfitting
GradientBoosting	0.5088	0.1696	0.3333	0.2248	Underfitting

Every one of the five classifiers lands on **exactly 0.5088** — the majority-class baseline to four decimal places — with macro F1 of 0.2248 and a lift of **+0.0 points**. All five are flagged **Underfitting**.

Why this is the strongest single table in the project

That is not five models failing. It is five models all discovering the same thing: in the original data `traffic_level` is statistically independent of `departure_hour`, so the best available strategy is to always predict the majority class — and each of them found it independently.

Macro F1 of 0.2248 is the giveaway. A model that predicts one class out of three scores about $\frac{1}{3}$ on macro recall no matter how respectable its accuracy looks, which is exactly why macro averaging is reported rather than weighted.

This is why Week 5 injected a rush-hour profile before the traffic classifier was allowed into the project, and why the accuracy reported in 11.7 measures that design decision rather than Indian roads. The comparison between these two tables is the honest disclosure.

11.11 Week 9 summary

Step	Result
1. Metrics	Best regressor <code>LinearRegression</code> : R^2 0.999707, RMSE ₹54.16, RSS 1.467e+07. <code>GradientBoosting</code> on <code>traffic_level</code> 57.0%; <code>LogisticRegression</code> on <code>cost_band</code> 85.6%
2. Overfit / underfit	“Good fit” throughout on the cost target — because it is a formula. Real overfitting appears on <code>cost_band</code> and the toll sub-model, where trees show gaps of 0.16–0.37
3. Validation	5-fold CV sd $\leq 4e-4$ on 25,000 rows, 4–20× wider on 1,140. Bootstrap 95% CI width 2.5e-06
4. Comparison	One split, one scoring module, one selection rule: error band → one-standard-error → simplicity
5. Tuning	2 of 3 searches improved. Ridge $\alpha \rightarrow 0$ reproduces Week 8 mechanically; <code>RandomForest</code> could not be improved; the traffic classifier gained 0.92 points
6. Advanced models	<code>RandomForest</code> , <code>AdaBoost</code> and <code>GradientBoosting</code> all evaluated; none beats a correctly specified linear model on cost

And the finding the checklist did not ask for. The regression above scores R^2 0.9997, but it is handed the true `toll_cost`, `parking_cost` and `fuel_consumption_litres` as input features — three of the four terms of the cost formula. A user supplies none of them. Part 12 takes them away.

Removing the magic numbers: a chained model pipeline

12.1 The problem

Up to Week 9, `app.py` served a prediction like this:

```
distance = haversine * 1.2355                    # constant
litres    = distance / mileage * TRAFFIC_MULT[traffic] # constant lookup
toll      = distance * 1.301                     # constant
parking   = 70.0                                # constant
cost      = cost_regressor([distance, mileage, price, toll, parking, litres, ...])
```

Only the last line is a model. Everything above it is arithmetic — and not incidental arithmetic, because it produces `fuel_consumption_litres`, the regressor's single strongest feature at correlation +0.96 with the target. The model was being handed three of the four terms of the cost formula and asked to add them up.

That reframes the headline. **R^2 0.999707 is a true number answering the wrong question:** “given the litres burnt and the toll paid, can you total the bill?” The user's question is “what will this trip cost?”, and they know neither.

Constant removed	Replaced by	Trained on
<code>WINDING_FACTOR = 1.2355</code> (the user had to know their mileage)	sub-model 1 — road distance	840 real OSRM road distances
<code>TRAFFIC_MULT = {...}</code>	sub-model 2 — mileage	generated data
<code>TOLL_RATE = 1.301</code>	sub-model 3 — litres	generated data
<code>DEFAULT_PARKING = 70.0</code> (already a model)	sub-model 4 — toll	generated data
	sub-model 5 — parking	fails; the mean is served instead
	sub-model 6 — traffic	generated data

12.2 The one piece of genuinely observed data

Everything in `road_trip_wide.csv` is generated. To fit a distance model that is not circular, Week 10 collected real data: `scripts/fetch_real_distances.py` samples city pairs stratified across six distance bands and asks the OSRM routing engine for the actual driving distance, appending each result to disk so a rate-limited run can resume without losing progress.

Measure	Value
Real routes collected	840
Straight-line range	40 – 1,248 km
Observed winding factor (road ÷ straight)	mean 1.2520 , sd 0.1448, range 1.0437 – 3.6015
The constant it replaces	<code>1.2355</code> – one number for that entire range

How wrong was the constant?

Measure	Value
Mean <i>signed</i> error	-3.3 km (-0.42%) – the bias
Mean <i>absolute</i> error	33.4 km (6.3%) – the per-route error
Worst overshoot	+155.3 km
Worst undershoot	-235.3 km

These two numbers say opposite-sounding things, and both matter

The constant is very nearly **unbiased**: averaged over 840 routes it is within half a percent of the truth. That is exactly why it survived review for so long.

But *unbiased on average* is not *correct*. Route by route it is off by **6.3%** typically, and by tens of percent at the extremes. **A user does not take the average of every road trip in India. They take one specific trip, and on that trip the errors do not cancel.**

The extreme routes are real, and they are the whole argument

Route	Straight line	Real road	Factor
Surat → Bhavnagar	94.2 km	339.4 km	3.60
Valsad → Bhavnagar	151.6 km	406.7 km	2.68
Badlapur → Bhavnagar	312.3 km	600.8 km	1.92
Moradabad → Chanduasi	43.1 km	44.9 km	1.04

A factor of 3.60 looks like bad data. It is not. **Surat and Bhavnagar face each other across the Gulf of Khambhat**: 94 km of straight line, 339 km of road around the head of the gulf. At the other end, Moradabad → Chanduasi is essentially a straight road.

No single multiplier can serve both. These points are **kept** in the training data rather than cleaned away, because they are precisely the cases a constant cannot represent and a model can. Note which model wins below: a **tree ensemble**, because it can isolate a region of the map. A linear fit gets dragged by these points instead of representing them.

12.3 Sub-model 1 — road distance

Two parameterisations of the same problem were compared, and the difference is instructive:

- **(a)** predict `road_km` directly
- **(b)** predict the winding *factor*, then multiply by haversine

(b) removes trip length from the target, so the model spends its capacity on the part that is genuinely unknown — the *shape* of the route — instead of re-learning “longer straight line, longer road”. It is the same idea as Week 6’s interaction term: hand the model the quantity that actually means something.

Target	Model	Test R ²	RMSE (km)	MAE (km)	CV mean	CV sd
(a) direct km	LinearRegression	0.985130	53.96	34.61	0.986733	0.0016
(a) direct km	Ridge (alpha=1)	0.981542	60.12	40.31	0.983237	0.0033
(a) direct km	DecisionTree	0.976503	67.83	44.33	0.974611	0.0071
(a) direct km	RandomForest (300)	0.985008	54.18	34.85	0.985587	0.0031
(a) direct km	AdaBoost	0.979909	62.72	46.71	0.979791	0.0034
(a) direct km	GradientBoosting	0.987542	49.39	33.86	0.988248	0.0026
(b) factor × haversine	LinearRegression	0.985008	54.18	34.47	0.986450	0.0020
(b) factor × haversine	Ridge (alpha=1)	0.985297	53.66	34.19	0.986705	0.0018
(b) factor × haversine	DecisionTree	0.978289	65.20	42.10	0.978193	0.0051
(b) factor × haversine	RandomForest (300)	0.988912	46.60	30.67	0.990592	0.0020
(b) factor × haversine	AdaBoost	0.983019	57.66	40.82	0.984999	0.0014
(b) factor × haversine	GradientBoosting	0.988827	46.77	31.28	0.988979	0.0020

	R ²	RMSE (km)	MAE (km)
BASELINE — haversine × 1.2355	0.9856	53.13	34.33
CHOSEN — RandomForest (300) (factor x haversine)	0.9889	46.60	30.67

The model beats the constant by 10.7% on MAE (34.33 → 30.67 km) and about 12% on RMSE. Real, and deliberately not oversold: endpoint coordinates only partly determine route shape, because the rest depends on where roads and bridges happen to be — information that is not in the features.

Parameterisation (b) wins, exactly as predicted. **A sub-model that could not beat the constant it replaces would not have earned its place**, which is why every table in this part carries the constant as its baseline row.

12.4 Sub-models 2 to 6

Sub-model 2 — mileage (vehicle + fuel → km/l)

Model	Test R ²	RMSE (km/l)	Train R ²	CV sd	Verdict
LinearRegression	+0.673405	1.810	0.687806	0.0019	Good fit
Ridge (alpha=1)	+0.673423	1.810	0.687806	0.0019	Good fit
DecisionTree	+0.673405	1.810	0.687806	0.0019	Good fit
RandomForest (300)	+0.673433	1.810	0.687805	0.0019	Good fit
AdaBoost	+0.665431	1.832	0.679392	0.0021	Good fit
GradientBoosting	+0.673418	1.810	0.687806	0.0019	Good fit

Chosen: LinearRegression

Sub-model 3 — litres (replaces TRAFFIC_MULT)

Model	Test R ²	RMSE (litres)	Train R ²	CV sd	Verdict
LinearRegression	+1.000000	0.000	1.000000	0.0000	Good fit
Ridge (alpha=1)	+0.999913	0.228	0.999911	0.0000	Good fit
DecisionTree	+0.999892	0.255	1.000000	0.0003	Good fit
RandomForest (300)	+0.999738	0.397	0.999988	0.0002	Good fit
AdaBoost	+0.974013	3.950	0.975193	0.0013	Good fit
GradientBoosting	+0.999687	0.434	0.999819	0.0000	Good fit

Chosen: LinearRegression

Sub-model 4 — toll (replaces TOLL_RATE = 1.301)

Baseline (the constant it replaces): R² +0.7950, RMSE 212.42 ₹

Model	Test R ²	RMSE (₹)	Train R ²	CV sd	Verdict
LinearRegression	+0.794912	212.474	0.804551	0.0073	Good fit
Ridge (alpha=1)	+0.794959	212.450	0.804545	0.0073	Good fit
DecisionTree	+0.610866	292.675	0.985595	0.0199	Overfitting
RandomForest (300)	+0.711665	251.933	0.950945	0.0159	Overfitting
AdaBoost	+0.783347	218.383	0.797783	0.0089	Good fit
GradientBoosting	+0.792275	213.836	0.813913	0.0076	Good fit

Chosen: LinearRegression

Sub-model 5 — parking (replaces DEFAULT_PARKING = 70)

Baseline (the constant it replaces): R² -0.0178, RMSE 50.51 ₹ · **Baseline (train mean):** R² -0.0010, RMSE 50.09 ₹

Model	Test R ²	RMSE (₹)	Train R ²	CV sd	Verdict
LinearRegression	-0.000911	50.085	0.000080	0.0018	Underfitting
Ridge (alpha=1)	-0.000911	50.085	0.000080	0.0018	Underfitting
DecisionTree	-1.004573	70.880	0.988768	0.0304	Overfitting
RandomForest (300)	-0.260242	56.201	0.820583	0.0105	Overfitting
AdaBoost	-0.001358	50.097	0.000606	0.0015	Underfitting
GradientBoosting	-0.006166	50.217	0.020462	0.0035	Underfitting

Chosen: train mean (no model beat it)

Sub-model 6 — traffic level (classifier)

Model	Accuracy	Precision	Recall	F1	Baseline	Lift (pts)	Verdict
LogisticRegression	0.5456	0.5393	0.5634	0.5373	0.3766	+16.9	Good fit
DecisionTree	0.5604	0.5639	0.5696	0.5641	0.3766	+18.4	Good fit
RandomForest (300)	0.5604	0.5639	0.5696	0.5641	0.3766	+18.4	Good fit
AdaBoost	0.5692	0.5718	0.5795	0.5718	0.3766	+19.3	Good fit
GradientBoosting	0.5696	0.5729	0.5797	0.5724	0.3766	+19.3	Good fit

Chosen: GradientBoosting

Sub-model 2, mileage — a structural ceiling, not a modelling failure

Test R^2 **0.6734**, and every model family lands on the same number to three decimals. That is the correct answer: the generator draws mileage *uniformly* inside a per-(vehicle, fuel) range, so the best any model can do is name the middle of the right cell, and the residual is uniform noise by construction.

Two details worth knowing. The nine (vehicle, fuel) cells cannot be reproduced by additive one-hots alone — $1 + 2 + 2 = 5$ parameters cannot fit nine independent means — so the feature set includes the four vehicle $\times$ fuel **interaction terms**, and with them the design matrix spans all nine cells exactly. And this sub-model changes the *interface*: the app used to default every car to 15.5 km/l, and now predicts 19.5 for a diesel hatchback and 10.8 for a CNG SUV.

Sub-model 3, litres — reading the generator off the coefficients

Test $R^2 = 1.000000$. The model has not approximated the data-generating rule; it has reproduced it. That claim is worth cashing in, because $\text{litres} = (\text{distance} / \text{mileage}) \times \text{traffic_multiplier}$ is a *product*, and an additive model cannot express a product. Handed the ratio and its interactions with the traffic one-hots, the coefficients become the multipliers themselves:

Traffic level	Recovered from the fit	Week 4 value	Error
Low	0.944000	0.9440	-0.000000
Medium	1.045000	1.0450	-0.000000
High	1.165000	1.1650	-0.000000

Intercept $+2.61\text{e-}07$ — zero, because the formula has no constant term. **Six decimal places.** This is the Week 6 lesson in its purest form: the features were the whole problem, and with the right ones a linear model *is* the generating rule.

Sub-model 4, toll — and the first honest overfitting in the project

Test R^2 **0.7949**, RMSE ₹212, and the fitted model does not beat the constant it replaces (R^2 0.7950 vs 0.7949). The generator sets `toll = distance × N(1.301, 0.360)`, so the *rate* is recoverable but the per-trip spread is injected noise. That RMSE is **irreducible on this data**, and a model that appeared to beat it would be leaking.

Look at the trees in that table, though — this is the first regression target in the project with real noise, and they behave exactly as theory says. `DecisionTree` reaches train R^2 0.9856 against test 0.6109: a gap of 0.37, correctly flagged **Overfitting**. Week 9's step 2 was uninformative on the deterministic cost target; here it identifies the wrong model immediately.

So the toll sub-model is kept for a different reason than accuracy: it is fitted, it reports its own error bar, and it is evaluated like everything else instead of being asserted.

Sub-model 5, parking — a negative result, reported

Every one of the six model families scores test $R^2 \leq 0$ — worse than predicting the mean. `DecisionTree` reaches R^2 -1.0046 with train R^2 0.9888: pure memorisation of noise, and the cleanest overfitting example in the project.

This is the correct answer, not a bug. The generator draws parking uniformly from {0, 40, 60, 80, 120, 150} **independently of every other column**, so it carries no signal at all. A model with negative R^2 has no business in a prediction path, so the app serves the training mean (₹75.09) and the negative result is disclosed through `/api/metrics`.

Worth noting: the old hardcoded ₹70.0 was not even the mean of the training data.

12.5 Chaining them properly — stacking, and a bug worth measuring

Six sub-models feeding a seventh introduces a failure mode that is easy to miss.

The cost regressor was trained on the **true** toll, parking and litres. If it is then served **predicted** ones, it has learned to trust inputs that are exact and receives inputs that are not. That is a **train/serve mismatch**, and rather than assume it matters, both ways are measured.

Building the training components correctly needs care too. Sub-model predictions on their own training rows are optimistic — the model has already seen those rows — so the component features for training come from `cross_val_predict` (**out-of-fold**): no row's features were produced by a model that had seen that row. Test rows use the sub-models fitted on the full training split. That is textbook **stacking**.

Architecture	Test R^2	RMSE (₹)	MAE (₹)
A. fed true toll / parking / litres (Week 6)	0.999707	54.16	39.66
B. chained, trained on true / served predicted	0.964602	595.37	390.56
C. chained, trained on predictions — shipped	0.964542	595.88	391.16
D. end-to-end, user inputs only	0.850960	1221.67	644.65

Reading the four rows — this is the centre of the project

A → C is the honest headline. R^2 falls by 0.0352 and MAE rises from ₹39.66 to ₹391.16 — roughly ten times larger. That gap is not a regression in quality; it is the price of the model deriving its own inputs instead of being handed them. **C is what a user actually receives, so C is what the app now reports.**

B vs C went the opposite way to expectation. B (trained on truth, served predictions) scores MAE ₹390.56; C (trained on its own predictions) scores ₹391.16 — B is *marginally better*. The mismatch cost essentially nothing, and the reason is specific: the sub-model errors are close to **zero-mean**, so the linear cost model's coefficients are almost unchanged by refitting on them.

C still ships, because it is the correct construction and the property that rescues B is a fact about this dataset rather than something to rely on. But the honest report is that **on this data the mismatch did not bite**, and claiming otherwise would be inventing a result.

12.6 Where the remaining error comes from

“The honest model is worse” is not a finding. “*Toll and parking noise account for nearly all of it*” is. Decomposing the test-set error into its sources:

Source	Std deviation (₹)	Could a better model fix it?
fuel bill, via litres	479.22	Yes — it inherits the distance error
toll (injected noise)	218.78	No — drawn from $N(1.301, 0.360)$ by construction
parking (uniform noise)	50.06	No — drawn uniformly at random
quadrature sum of the three	529.17	
chained model RMSE	595.88	

The quadrature sum of the irreducible terms is **₹529** against a model RMSE of **₹596** — a ratio of 1.13. The two agree closely enough to say the chain sits near the floor this dataset allows, and that the residual is **the data's unpredictability rather than the model's weakness**. The distance error is 44.42 km, which propagates into the fuel bill — the only term a better model could still improve.

There is a corresponding test in the suite (`test_metrics_error_budget_explains_the_remaining_error`) asserting that ratio stays between 0.7 and 1.6, so a future regression in the chain shows up as a failing test rather than a quietly worse number.

12.7 The end-to-end model — and where Week 6's lesson reverses

Architecture D removes even the chain. One model, fed **only** what a traveller could know: two city names (as real latitude/longitude), vehicle, fuel type, departure hour, month, passengers, mileage and fuel price. No distance, no toll, no litres, no parking.

It also re-runs Week 6's experiment in a harder setting. The true cost contains `(distance / mileage) × fuel_price`, so two feature sets are compared: the raw inputs, and the raw inputs plus `haversine/mileage` and `(haversine/mileage) × fuel_price`.

Model and feature set	Test R^2	MAE (₹)
Linear, base features	0.656346	1118.68
Linear, + 2 engineered columns	0.676689	1051.63
RandomForest (300), base features	0.850960	644.65

Source of the improvement	MAE saved (₹)
Two engineered columns	67.05
Switching to a 300-tree RandomForest	474.03

Week 6's claim does not generalise — and that is the interesting part

Week 6 argued, and demonstrated, that **domain understanding beat model complexity**. End-to-end the result **reverses**: the ensemble wins by a factor of about 7.

This is not a contradiction of Week 6, it is a **boundary condition** on it, and the reason is identifiable. When distance was *given*, the only thing the linear model could not express was a product — so handing it the product fixed the one gap. End-to-end, the model must first infer road distance from four coordinates, and that is a genuinely non-linear function of geography: coastlines, mountains, the Gulf of Khambhat. No finite set of hand-made interaction terms captures a map, and partitioning space is exactly what trees do.

Feature engineering beats model complexity when the missing structure is something you can write down. When the missing structure is a map, it does not.

What the end-to-end model leans on

Feature	Share of importance
log_haversine	37.03%
haversine_km	32.18%
mileage	11.19%
fuel_price	2.91%
abs_dlat	2.23%
abs_dlon	1.86%
dest_lat	1.74%
start_lat	1.72%
mid_lon	1.67%
mid_lat	1.55%

One trap in this ranking

Distance dominates: haversine and its log together carry **69%** of the prediction, which is right — cost is mostly fuel, and fuel is mostly distance. But the vehicle and fuel one-hots score **below 0.1%**, and it would be wrong to conclude vehicle type does not matter.

Week 4 recovered a real per-km maintenance rate that differs by vehicle (Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 ₹/km) and Week 6 showed that dropping those columns made the app quote an identical price for an SUV and a hatchback. The rate spread is about 0.27 ₹/km — roughly ₹108 on a 400 km trip — and end-to-end that signal sits *underneath* the toll and parking noise, which is hundreds of rupees.

The importance is low because the noise here is large, not because the effect is absent. It is precisely why the app serves the chained pipeline: pin distance and litres down first, and the vehicle term becomes visible again. `passengers`, at 0.33%, is the genuine null — the recovered formula has no passenger term, so anything above zero there is the forest fitting noise.

12.8 Week 10 summary

	Result
Constants removed from <code>app.py</code>	5
Sub-models fitted	6 — one trained on real observed data
Sub-models that failed	1 — parking, $R^2 \leq 0$, reported as a negative result
Real data collected	840 OSRM road distances
Distance model vs its constant	MAE 34.33 → 30.67 km (−10.7%)
Honest cost accuracy	R^2 0.9645, MAE ₹391 (was R^2 0.9997, MAE ₹40 with inputs handed in)
Error budget	₹529 of the ₹596 RMSE is irreducible noise in the data
Tests	65, all passing

The three things Week 10 established

1. **A near-perfect score is a question about the question.** R^2 0.9997 was true and uninformative; the number that describes a user's prediction is 0.9645.
2. **A model that fails is a result.** Six model families could not predict parking, because nothing in the data predicts parking. Reporting that is worth more than tuning until it looks respectable.
3. **“Features beat complexity” has a boundary.** It held when the missing structure was a product you could write down. It reversed when the missing structure was a map.

The application — API, frontend, tests, deployment

13.1 What happens on one prediction

This is the whole system in one sequence. Worth being able to narrate.

```
POST /api/predict { start_city, destination_city, vehicle_type, fuel_type,
                    departure_hour, month, passengers,
                    mileage?, fuel_price?, parking_cost?, traffic_level? }
```

1. resolve_city() city name -> (state, lat, lon), largest city wins on ambiguity
2. road_distance_km() try LIVE OSRM routing
on failure -> sub-model 1 (distance model)
3. predict_traffic() sub-model 6: hour + month -> Low / Medium / High
(skipped if the user supplied traffic_level)
4. default_mileage() sub-model 2: vehicle + fuel -> km/l
(skipped if the user supplied mileage)
5. fuel_price_for() per-state lookup table; a user-supplied price always wins
6. predict_litres() sub-model 3: distance + mileage + traffic -> litres
7. predict_toll() sub-model 4: distance -> Rs
8. DEFAULT_PARKING sub-model 5 FAILED, so the training mean is used
9. cost_model_chained the chained regressor totals it
10. predict_band() cost-band classifier -> Excellent / Good / Average / Poor
11. warnings[] extrapolation, estimated fuel price, routing fallback
12. breakdown fuel / toll / parking / wear-and-tear, by subtraction

13.2 Worked example

Jaipur → Agra, SUV, Diesel, 9am departure, nothing else supplied:

Field	Value	Where it came from
distance_km	240.05	osrm — live routing
mileage	12.68 km/l	mileage_model — sub-model 2
fuel_price	₹102.27	state table, marked estimated
traffic_level	High	inferred by sub-model 6 from 9am
fuel_consumption_litres	22.055 L	sub-model 3
toll_cost	₹311.24	sub-model 4
parking_cost	₹75.09	training mean — no model beat it
total_trip_cost	₹2,879.60	chained cost regressor
typical_error	± ₹391.16	the chain's own held-out MAE
cost_band	Poor	cost-band classifier

Breakdown: fuel ₹2,255.56 + toll ₹311.24 + parking ₹75.09 + wear ₹237.71 = ₹2,879.60 — it sums exactly, which is asserted by a test.

13.3 The API

Endpoint	Purpose
GET /api/cities?q=	Autocomplete over the 537 in-domain cities. Prefix matches rank above substring matches; each result carries its state's fuel prices so the UI can prefill without another round trip.
GET /api/default-mileage	What the mileage sub-model predicts for a vehicle + fuel pair. Lets the interface start from a modelled figure instead of one hardcoded 15.5 for every car.
POST /api/predict	The chained prediction, with breakdown, band, warnings and error bar.
POST /api/departure-sweep	Expected cost for all 24 departure hours — “when should I leave?”
GET /api/metrics	Honest test-set numbers: <code>served</code> , <code>fed_true_components</code> , the <code>honesty_gap</code> between them, per-sub-model metrics, and the error budget.
GET /api/loss-curve	The Week 3 gradient-descent loss curve and the sklearn comparison, including the Week 8 non-convergence.
GET /api/health	Liveness.

The design decision worth defending: you are only asked what you could know

The model's strongest feature is `fuel_consumption_litres` at correlation +0.96. It would be trivially easy to build a form that asks for it — and the resulting product would be useless, because **nobody knows how many litres they will burn before they set off**. The same is true of toll and, for most people, their car's exact mileage.

So the form asks for two city names, a vehicle, a fuel type and a time. Everything else is predicted. That choice is *why* the honest accuracy is R^2 0.9645 rather than 0.9997 — and it is the right trade, because the 0.9997 version could not be used by a real person.

Two things the API deliberately does *not* hide

- **The fuel price is prefilled but editable.** The per-state table is verified for only 6 of 35 states. Anyone filling their own tank knows the real figure better than an estimate does, so a supplied value always wins and is reported as `source: "user"` — which correctly suppresses the estimated-price warning, because with a real figure there is nothing left to caveat.
- **Warnings are attached per prediction.** Beyond 1,500 km the response says it is extrapolating. If OSRM was unreachable, it says the distance came from the model. If the fuel price was an estimate, it says so and invites the user to override it.

The departure sweep — expected cost, not argmax cost

Worth understanding because it is the one place a classifier's *probabilities* are used rather than its label:

$$E[\text{cost} \mid \text{hour}] = \sum_{\text{levels}} P(\text{level} \mid \text{hour}) \times \text{cost}(\text{level})$$

Taking the single most likely traffic level would collapse 24 hours onto 3 distinct values and throw away everything the model knows about its own confidence. Weighting by probability gives a smooth curve where an hour that is 55% High reads differently from one that is 95% High.

13.4 The frontend

Aspect	Detail
Stack	React 19, Vite 7, Tailwind v4, GSAP for scroll animation, Recharts for the charts
Style	Editorial single-page site — photographic hero, torn-paper edges, serif display type
Components	17, including <code>Predictor</code> , <code>ResultCard</code> , <code>Breakdown</code> , <code>DepartureChart</code> , <code>LossCurve</code> , <code>ReportCard</code> , <code>Warnings</code>
Build	<code>npm run build</code> emits into <code>static/</code> , which FastAPI serves at <code>/</code> — so production is one process
Mileage field	Seeded from <code>/api/default-mileage</code> for the chosen vehicle and fuel; stops updating once the user moves the slider

13.5 The test suite — 65 tests

The tests deliberately cover the things that *broke* during development rather than trivially exercising every endpoint. A selection, with what each one protects:

Test	What would break without it
<code>test_litres_come_from_the_model_and_still_match_the_recovered_formula</code>	Sub-model 3 silently drifting away from the Week 4 formula
<code>test_toll_is_a_model_and_not_a_single_multiplication</code>	A regression back to <code>distance × 1.301</code> — it asserts the implied rate <i>varies</i> with distance
<code>test_vehicle_type_actually_changes_the_prediction</code>	The real bug from Week 6: the UI collected <code>vehicle_type</code> and the model ignored it, so all three vehicles returned an identical total
<code>test_distance_model_beats_the_constant_on_the_gulf_of_khambhat</code>	The distance sub-model regressing to something no better than the constant on the hardest real route
<code>test_the_error_bar_beside_a_prediction_describes_that_prediction</code>	Re-introducing the ₹39.66 error bar that no real prediction can achieve
<code>test_metrics_error_budget_explains_the_remaining_error</code>	The chain getting quietly worse — it pins the RMSE-to-irreducible-noise ratio to 0.7–1.6
<code>test_metrics_disclose_the_parking_negative_result</code>	The failed sub-model being quietly dropped from the report
<code>test_offline_the_distance_model_takes_over_from_osrm</code>	The fallback path breaking unnoticed — it monkeypatches the network away
<code>test_night_departure_is_cheaper_than_rush_hour</code>	The traffic classifier losing its effect on cost
<code>test_breakdown_sums_to_the_total</code>	A breakdown that does not add up to the number shown
<code>test_ambiguous_city_name_resolves_to_the_largest</code>	India has several Udaipurs; it must pick Rajasthan's, not Tripura's

13.6 Deployment

A two-stage `Dockerfile`: Node compiles the React app, then the Python image serves both the API and the built site, so **one container runs everything**. `render.yaml` describes the service for Render's free tier; the health check is `/api/health`.

- **Single worker, deliberately.** The models and the 3,739-city table load per process, and the OSRM distance cache is in-process. Extra workers multiply memory and cold routing calls for no gain at this traffic level.
- **The image is lean.** `.dockerignore` excludes the notebook, raw CSVs, scripts, tests, frontend source and the two large evaluation-only model files — only `app.py`, `roadtrip_features.py`, two small data files, the two served models and the built site ship.
- **Hosts that inject \$PORT** (Render, Railway, Fly, Cloud Run) are handled; it defaults to 8000 otherwise.
- **Free tiers sleep.** Render's free instance spins down when idle and the first request after that takes ~30 s.

Stated plainly: the Docker build has *not* been executed — Docker is not installed in the development environment. The `--outDir` override the image depends on was verified directly, but expect to iterate once on the first real build. Saying this is better than implying it was tested.

Bugs found in this project's own code, and limitations

These are worth reading closely. An examiner who asks “what went wrong?” is offering the easiest marks in the viva, and a project that can name its own mistakes reads as far more credible than one that reports only successes.

14.1 Five real bugs, and how each was caught

1. The error bar next to every prediction was unachievable

Field	Detail
Symptom	<code>/api/predict</code> returned <code>typical_error: 39.66</code> beside every estimate.
Cause	That was the MAE of the cost regressor measured <i>with</i> the true toll, parking and litres handed in as features. A user supplies none of them, so no real prediction could ever achieve it .
Fix	It now reports the chained pipeline's own held-out MAE, ₹391.16, plus a note saying most of it is toll and parking.
Guard	<code>test_the_error_bar_beside_a_prediction_describes_that_prediction</code> asserts it tracks the served figure and <i>exceeds</i> the fed-components one.

This was the single least honest thing in the project. It is also the most instructive: the model was never wrong, the *reporting* was.

2. The model-selection rule compared R^2 with a relative tolerance

Field	Detail
Symptom	<code>pick_best</code> chose a Ridge fit at 53.7 km RMSE over a random forest at 46.6 km — 15% worse — because the Ridge fit had a marginally steadier CV score.
Cause	It treated two models as tied if their R^2 was within 0.5% <i>relative</i> . Near $R^2 = 0.99$ that band is enormous in error terms.
Fix	Closeness is now judged on the error (2% of the best RMSE, or 0.5 accuracy points), then the one-standard-error rule decides, then simplicity.

3. The scaler leaked test-set information into training

Field	Detail
Symptom	None visible — which is what makes it dangerous.
Cause	<code>MinMaxScaler</code> was fitted on the <i>whole</i> matrix before <code>train_test_split</code> , so test-set minima and maxima influenced the transformation applied to the training data.
Fix	Every model is now wrapped in <code>Pipeline([MinMaxScaler, model])</code> , so cross-validation re-fits the scaler on each training fold and no validation-fold information reaches it.
Scale of the problem	Small for <code>MinMaxScaler</code> on this data — but it is exactly the mistake cross-validation exists to catch, so it was fixed rather than excused.

4. A 227 MB model file

Field	Detail
Symptom	<code>models/pipeline.joblib</code> came out at 227 MB , replacing a 10 KB file. The end-to-end benchmark was 546 MB.
Cause	A 300-tree <code>RandomForest</code> fitted to 20,000 rows pickles to about 210 MB, and almost all of it was <code>distance_dataset_model</code> — a model used <i>only</i> inside the offline evaluation.
Fix	Compression alone was not the answer. The right question was <i>which models does the API actually load?</i> Splitting the evaluation-only models into a separate, git-ignored file and compressing both gives 5.1 MB served — a 44× reduction.

5. pandas 3 broke scikit-learn's cross-validation

Field	Detail
Symptom	<code>TypeError: only integer scalar arrays can be converted to a scalar index</code> , thrown from inside <code>cross_val_score</code> .
Cause	pandas 3 stores text columns as Arrow-backed arrays (<code>ArrowStringArray</code>), and those do not support the fancy indexing (<code>y[train_idx]</code>) that scikit-learn performs internally.
Fix	One helper, <code>evaluation.labels()</code> , converts label columns to a plain NumPy object array once, so every call site is correct rather than being debugged per model.

14.2 Limitations, stated plainly

1. **The wide dataset is generated, not observed.** It is physically consistent with the original data — same recovered formula, real geography, real coordinates — but it is not independent evidence. The exception is `data/real_distances.csv`: 840 genuinely observed road distances, which is what the distance sub-model is trained on.
2. **The generated geography is off by 6.3% per route.** The wide dataset sets `distance_km = haversine × N(1.2355, 0.082)`. Against the 840 real routes that is nearly unbiased in the mean (−0.4%) but wrong by 6.3% on a typical single route. Costs measured *on* that dataset stay self-consistent, but the distance model is deliberately trained on the real data instead.

3. **Fuel prices: 6 of 35 states verified, 29 estimated.** No free, current, machine-readable state-wise feed exists for India; three candidate sources were checked and rejected (one stale, one paid, one JavaScript-only). The CSV records which is which per row, the API warns when a prediction uses an estimate, and the user can override it.
4. **The rush-hour traffic profile was injected deliberately.** In the original data traffic is statistically independent of departure hour — all five classifiers land on exactly the 50.88% majority baseline. The generator adds a realistic profile, so the traffic classifier's 57.0% measures that design choice, not Indian roads.
5. **Parking is not predicted, because it cannot be.** Six model families all scored test $R^2 \leq 0$, so the training mean is served. Roughly ₹50 of the reported error bar is this term alone.
6. **The error bar is large, and that is the honest number.** MAE ₹391, of which about ₹529 of the ₹596 RMSE is irreducible noise in the generated data. Only the fuel-bill term could be improved by a better model.
7. **Predictions beyond 1,500 km extrapolate.** The model was trained on 50–1,500 km; the API returns an explicit warning outside that range rather than silently guessing.
8. **Routing depends on a public demo server.** OSRM's free instance is rate-limited and makes no uptime promise. Distances are cached and fall back to the fitted distance model, and `distance_source` says which was used.
9. **Travel time is not modelled.** `TRAFFIC_MIN_PER_KM` remains a constant, deliberately: time is not part of the cost target, so fitting it would be inventing scope. It is labelled in the source rather than hidden.
10. **The Docker build has not been run.** See Part 13.6.

Viva preparation — 60 questions with answers

How to use this part

The questions are ordered from most to least likely. **Q1–Q12 are the ones that will almost certainly be asked** — if time is short, learn those and the three formulas in Q13, Q21 and Q30.

Two habits matter more than any single answer. First, **volunteer the honest number before being asked for it**: say “ R^2 0.9645 for a real prediction, and 0.9997 if the model is handed its own inputs” rather than waiting to be caught. Second, when you do not know something, **say what you would do to find out** — that answers the question behind the question.

Group A — the questions you will definitely get

Q1. What is this project, in one minute?

It predicts the cost of a road trip between any two of 537 Indian cities. You give it two city names, a vehicle, a fuel type and a departure time; it returns a cost estimate with a breakdown.

It is a regression problem — the target `total_trip_cost` is continuous — with two supporting classification tasks. It began from a 1,140-row dataset over 9 fixed city pairs. Over ten weeks it grew into a chained pipeline of six sub-models: road distance, fuel efficiency, litres burnt, toll, parking and traffic level are each predicted, and a final regressor totals them. The honest accuracy is **R^2 0.9645, MAE ₹391**.

Q2. Your R^2 is 0.9997. Isn't that suspiciously high?

Yes, and that is the most important thing I found. That 0.9997 was measured with the true `toll_cost`, `parking_cost` and `fuel_consumption_litres` handed to the model as input features. Those are three of the four terms of the cost formula, so the model was essentially being asked to add up numbers it had already been given.

It answers the question “*given the litres burnt and the toll paid, can you total the bill?*” A real user knows none of those before setting off. When the model has to derive them itself, the score is **R^2 0.9645, MAE ₹391** — and that is the figure the deployed API reports as `served`, with the larger one labelled `fed_true_components` beside it and the gap between them named.

Q3. Why is the error ₹391? That seems like a lot.

It is, and it is almost entirely the data's fault rather than the model's. I decomposed it:

- toll noise: $\pm$ ₹219 — the generator sets `toll` = distance $\times$ $N(1.301, 0.360)$, so the per-trip spread is random by construction
- parking noise: $\pm$ ₹50 — drawn uniformly from a fixed set, independent of everything
- fuel bill: $\pm$ ₹479 — this one inherits the distance prediction error

Added in quadrature that is **₹529**, against a model RMSE of **₹596** — a ratio of 1.13. The two agree closely enough to say the chain is near the floor this dataset allows. Only the fuel-bill term could be reduced by a better model; the other two are irreducible.

Q4. Explain gradient descent, and show that you implemented it yourself.

Three steps, repeated. **One:** compute the loss — mean squared error, $L = (1/n)\sum(\hat{y} - y)^2$. **Two:** compute the gradients, which point in the direction that *increases* the loss: $\partial L / \partial w = (2/n) X^T \cdot \text{error}$ and $\partial L / \partial b = (2/n) \sum \text{error}$. **Three:** step in the *opposite* direction — that is the “descent” — $w \leftarrow w - 1r \cdot \partial L / \partial w$.

I wrote it in pure NumPy, no sklearn, and all six gradients are computed in one matrix operation $X.T @ \text{error}$ rather than a Python loop per feature. It stops when the loss changes by less than $1e-8$. With learning rate 0.2 it converged at **epoch 14,317** out of a 50,000 budget.

Q5. Did your from-scratch version match sklearn?

Yes, to within **0.03%**. The largest disagreement on any parameter was **0.042** — four paise — against coefficients in the hundreds and thousands. Across all 1,129 trips the two models' predictions never differed by more than **₹0.011**, and R^2 was identical to six decimal places (0.992841).

And they had to match. The MSE loss for linear regression is convex — a bowl with exactly one minimum. sklearn jumps to the bottom algebraically by solving the normal equations; gradient descent walks down to the same bottom. Given a sensible learning rate and enough epochs there is nowhere else for either to land.

Q6. What would make them *not* match?

Four things, and I tested each rather than just listing them:

- **Learning rate too small** — at $lr = 0.001$ with a 3,000-epoch budget the weights were still about 2,025 away from sklearn's. The loss was still falling when the loop ended.
- **Too few epochs** — at $lr = 0.2$ the gap collapses from $\sim 1,260$ at 100 epochs to 0.94 at 10,000 to effectively zero by 50,000.
- **Learning rate too large** — above about $lr = 0.44$ on this data the updates overshoot the minimum, the loss oscillates and blows up to `inf`.
- **Unscaled features** — features on very different ranges turn the loss surface into a narrow valley, and one learning rate cannot suit every weight.

Q7. Why did you drop the one-hot city columns in Week 3? That looks like you were avoiding a hard case.

It was necessary, not convenient. In the original dataset every route is a *fixed pair* — Delhi only ever goes to Jaipur. So `start_city_Delhi` and `destination_city_Jaipur` contain **identical values in every row**.

That makes the design matrix **rank-deficient**: infinitely many different weight combinations give the exact same predictions. sklearn silently picks one using the pseudo-inverse; gradient descent, starting from zeros, converges on a different one. Both are correct, and the two weight vectors would look completely unrelated.

Since the whole point of Week 3 is to compare the two parameter sets, keeping those columns would have made a correct result *look* broken. Later weeks encode `vehicle_type` as one-hots, where there is no such degeneracy — and Week 6 shows dropping *that* was a real bug.

Q8. How did you know the dataset was synthetic?

I tested it rather than guessing. I subtracted the cost components I could compute exactly — fuel, toll, parking — and looked at the residual. If it were noise it would scatter around zero.

Instead it was **never negative, never zero**, and correlated **+0.85 with distance**. Dividing by distance gave a per-kilometre rate, and grouping that rate by each categorical column showed only `vehicle_type` split it — cleanly: Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 ₹/km. Fuel type, traffic and weather were all flat.

Reassembling the formula and evaluating it gives $R^2 = 0.999162$ **from arithmetic alone**, mean absolute error ₹18.44. That is not a model fit — it is the generating equation, recovered.

Q9. If the data is synthetic, what have you actually learned?

Three things, and I would give all three.

First, the discovery itself is a result. Recovering the generating formula to R^2 0.999162 required a hypothesis, a method and a test.

Second, the geography is real. Cities and coordinates come from GeoNames, and I collected **840 actual road distances** from the OSRM routing engine. The distance sub-model is trained on observed data and beats the hardcoded constant by 10.7% on MAE.

Third, it forced the project to be honest about which numbers mean what. Because I knew the target was a formula, I could recognise that a near-perfect R^2 was arithmetic rather than skill — which is exactly the reasoning that produced the two-number reporting in the API. A student who never noticed would have submitted R^2 0.9997 and believed it.

Q10. Walk me through Task 5.

Six steps, applied to every model in the project, all scored through one shared module so the notebook, the report and the API cannot disagree.

1. **Metrics** — RSS, RMSE, R^2 for regression; accuracy, precision, recall, F1 (macro-averaged) for classification.
2. **Overfit / underfit** — train vs test with stated thresholds: a gap above 0.05 is overfitting, both below 0.50 is underfitting.
3. **Validation** — 5-fold cross-validation on the training data, plus a 200-resample bootstrap.
4. **Comparison** — one table per family, on one common split.
5. **Tuning** — GridSearchCV over Ridge α , RandomizedSearchCV over RandomForest, GridSearchCV on the traffic classifier.
6. **Advanced models** — RandomForest (bagging), AdaBoost and GradientBoosting, in every table.

Best regressor: `LinearRegression`, R^2 0.999707, RMSE ₹54.16. Best classifiers: `LogisticRegression` on cost-band at 85.62%, and `GradientBoosting` on traffic at 56.96% against a 37.66% baseline.

Q11. Did hyperparameter tuning improve your model?

Two of the three searches improved, one did not — and the pattern is the point.

- **Ridge α** improved, but only against sklearn's default $\alpha = 1.0$, and it did so by driving the penalty to the bottom of the grid. The real finding is that this problem wants *no* regularisation.
- **RandomForest** did **not** improve. Twelve random draws over four hyperparameters could not beat 300 trees at their defaults.
- **The traffic classifier** improved genuinely, by +0.92 accuracy points, with `max_depth=4`.

The lesson: **tuning pays where the data is noisy and the model can overfit it**. Where the target is a formula and the features already span it, tuning has nothing to do — and a search reporting no improvement is evidence the model was specified correctly, not evidence the search was wasted.

Q12. Why grid-search Ridge if your best model is plain LinearRegression?

Because `LinearRegression` has no hyperparameter worth tuning — a grid search over it would be theatre. Ridge is the same model plus exactly one knob, so searching α is a real search whose answer means something.

And it had a specific hypothesis to test. In Week 8 I had rejected Ridge *by hand*, arguing from the Hessian condition number that regularisation fixes the conditioning at an unacceptable cost in accuracy. If that argument was right, the search should reach the same verdict mechanically without being told the theory. It did: the CV score is flat for every $\alpha \leq 0.01$ — where the penalty is too small to do anything, so Ridge is ordinary least squares — and falls monotonically above it, down to R^2 0.835 at $\alpha = 1000$.

Group B — concepts and definitions

Q13. Define RSS, RMSE, MAE and R^2 , and say when each is appropriate.

$RSS = \sum (y - \hat{y})^2$ — the raw quantity least squares minimises. It is a *sum*, so it grows with row count: comparable between models on the same split, meaningless across different splits.

$RMSE = \sqrt{RSS/n}$ — reads in the target's units. Squaring inside means one large error counts as much as many small ones, so quote it when large errors are what hurt.

$MAE = (1/n)\sum |y - \hat{y}|$ — average error, no squaring. The number to tell a user.

$R^2 = 1 - RSS/TSS$ — fraction of variance explained. Its weakness is exactly what this project hit: when the target is nearly deterministic, R^2 sits so close to 1 that differences that matter in rupees become invisible. In my regression table every model except AdaBoost scores $R^2 > 0.998$, yet the worst is seven times worse in RMSE.

That is why I read my tables from the RMSE column.

Q14. What is overfitting? How do you detect it, and did you see it?

Overfitting is when a model learns the training data's *noise* rather than its structure — so it scores well on data it has seen and badly on data it has not. Detection: compare train and test scores; a large gap is the signature.

I saw it in three places, and where I *didn't* see it is equally instructive:

- **cost_band classifier:** DecisionTree train 1.0000 vs test 0.7918 — a gap of 0.21. RandomForest 1.0000 vs 0.8354.
- **toll sub-model:** DecisionTree train R^2 0.9856 vs test 0.6109 — a gap of 0.37. This is the first regression target with real injected noise, and the tree fits it.
- **parking sub-model:** DecisionTree train 0.9888 vs test -1.00 — pure memorisation of pure noise, the cleanest example in the project.
- **The cost target:** DecisionTree reaches train $R^2 = 1.000000$ and *still* holds test 0.9986. Because the target is a formula, memorising and learning the rule give nearly the same predictions. So on that target the train/test check is close to uninformative — worth knowing, because it means a “good fit” verdict there is a property of the data, not proof of careful modelling.

Q15. What is underfitting? Show me an example from your project.

Underfitting is when the model is too simple, or the features carry too little signal, to capture the relationship — so it scores badly on *both* train and test.

My best example is on the original dataset. Predicting `traffic_level` from `departure_hour`, all five classifiers land on **exactly 0.5088** — the majority-class baseline to four decimal places — with macro F1 of 0.2248 and a lift of +0.0 points. All five are flagged underfitting.

That is not five models failing. It is five models all discovering the same thing: in that data traffic is statistically independent of hour, so the best available strategy is to always predict the majority class. The macro F1 of 0.22 is the giveaway — predicting one class out of three scores about a third on macro recall however good the accuracy looks.

Q16. Explain cross-validation, and why the standard deviation matters more than the mean.

K-fold splits the training data into k parts, trains on k-1 and validates on the remaining one, k times. You get k scores.

The mean is a better estimate than a single split, because a single split's score depends on which rows happened to land in the test set. But the **standard deviation across folds** answers a different and often more useful question: *is this model stable, or did it get lucky?* A model with a high mean and a wide spread is unreliable — it is sensitive to which particular rows it saw.

In my project the spreads on 25,000 rows are $1e-5$ to $4e-4$ — so I say plainly that **cross-validation confirmed stability but did not discriminate between models**. It earns its place on the original 1,140-row dataset, where the same models and the same code give spreads **4 to 20 times wider**. That contrast is the clearest argument for why cross-validation is on the checklist at all: with one split you would never see it.

Q17. What is the bootstrap, and how does it differ from k-fold?

Both estimate variability, but of different things. **K-fold** asks “how does the score vary across disjoint held-out folds?” **Bootstrap** asks “how would the score vary if I had drawn a different training sample of the same size?” — so it resamples the training rows *with replacement*, refits, and scores each fit on a fixed test set.

I ran 200 resamples on the best regressor: mean R^2 0.999707, standard deviation $1e-6$, 95% interval [0.999705, 0.999708] — a width of about $3e-6$. The fit does not depend in any meaningful way on which rows were drawn.

Q18. Why macro-averaged precision and recall, not weighted?

Macro weights every class equally; weighted weights by class frequency. With an imbalanced target, weighted averaging can **hide a class the model never predicts at all** — you get a respectable score while one class is being completely ignored.

My project contains the proof. On the original dataset the traffic classifiers score 50.88% accuracy, which sounds passable. Their macro F1 is **0.2248**, which immediately reveals they are predicting a single class out of three. Macro averaging is what makes that visible.

Q19. Why is one-hot encoding correct here and not label encoding?

Label encoding would map Hatchback→0, Sedan→1, SUV→2. That asserts an *ordering* and a *spacing* — that an SUV is “twice” a Sedan, and that Sedan sits exactly halfway between the other two. None of that is meaningful for a nominal category, and a linear model would take the arithmetic literally.

One-hot gives each category its own independent column and asserts no ordering. I use `drop_first=True` to avoid the **dummy variable trap**: if you keep all three vehicle columns they sum to 1 in every row, which is perfectly collinear with the intercept and makes the design matrix singular.

Label encoding *would* be right for a genuinely ordinal variable — and `traffic_level` (Low < Medium < High) is arguably one. I still used one-hot there for consistency, which is a defensible choice rather than the only one.

Q20. Why MinMaxScaler rather than StandardScaler?

MinMax maps to a bounded 0–1 range: $(x - \min)/(\max - \min)$. Standardisation subtracts the mean and divides by the standard deviation, giving unbounded values centred on zero.

I chose MinMax for two reasons. It suits **gradient descent** — all partial derivatives land on a comparable scale, so a single learning rate works for every weight. And it makes the coefficients **readable**: because every feature spans exactly 0–1, a weight is literally “how many rupees the cost changes as this feature moves from its minimum to its maximum”, which is why I could immediately identify `fuel_consumption_litres` at +2938 as the dominant driver.

StandardScaler would be the better choice if the features were roughly Gaussian and I cared about outlier robustness — MinMax is sensitive to extreme values, which is part of why the outlier step comes first.

Q21. Why did you scale the features but not the target?

Deliberately, so the learned coefficients stay in rupees and remain interpretable. Scaling the target would leave the model equally accurate and completely unreadable — every weight would be in arbitrary units and I could not tell an examiner what any of them meant.

Scaling the target is worth doing when the optimiser struggles with the target's magnitude, or when comparing across differently-scaled targets. Neither applied here.

Q22. What is bagging? What is boosting? Which did better for you?

Bagging (RandomForest) trains many trees *in parallel* on bootstrap samples of the rows, each considering a random subset of features, then averages them. It attacks **variance**: individual deep trees overfit, and averaging many decorrelated ones cancels much of that out.

Boosting (AdaBoost, GradientBoosting) trains weak learners *sequentially*, each one focused on what the previous ones got wrong — AdaBoost by re-weighting misclassified rows, GradientBoosting by fitting the residuals. It attacks **bias**.

In my project RandomForest was the strongest ensemble on most targets and GradientBoosting won the traffic classification. **AdaBoost was consistently the weakest**, and I can say why: it fits shallow stumps and chases the rows it got wrong, which on a smooth additive target with injected noise means spending its capacity on the noisiest rows instead of representing the smooth part.

The headline, though, is that **none of the three beat a correctly specified linear model on the cost target** — ₹54 RMSE for linear against ₹87 for RandomForest and ₹399 for AdaBoost.

Q23. Why did a linear model beat a Random Forest? Isn't that unusual?

Not when the underlying relationship really is linear-plus-a-known-product, which is exactly this case. The cost is a *sum* of terms, one of which is a product of two features. Once I supply that product as a column, a linear model can represent the target **exactly**.

A tree ensemble has to *approximate* the same relationship with axis-aligned splits. It gets close, but it can never be exact, and it pays in variance for the flexibility it does not need.

The general principle: **match the model to the structure of the problem**. Trees win when the structure is something you cannot write down — which is precisely what happens in my end-to-end model, where they beat the linear one by a factor of seven because the missing structure is a *map*.

Q24. What is regularisation? Why did you reject it?

Regularisation adds a penalty on the size of the weights, trading a little bias for a reduction in variance. Ridge (L2) minimises $\|Xw - y\|^2 + \alpha\|w\|^2$, giving $w = (X^T X + \alpha I)^{-1} X^T y$ — and the αI term is exactly what makes the matrix better conditioned. Lasso (L1) penalises absolute values instead and can drive weights to exactly zero, so it also does feature selection.

I rejected it because I measured what it cost. Ridge did fix my conditioning problem — condition number from 16,155 down to 263 — and gradient descent then converged exactly. But test MAE went from **₹39.66 to ₹255.42**, six times worse.

The reason is that I *do not need* gradient descent. The normal equations solve this model exactly in one step and are indifferent to the condition number. So regularising would trade real accuracy to fix a difficulty that only exists because of the solver I chose. Week 9's grid search confirmed it independently by driving α to the bottom of the grid.

Q25. What is multicollinearity? Did you have it?

Multicollinearity is when features are strongly linearly related to each other. The fit stays valid but the individual coefficients become unstable — many different weight combinations explain the data almost equally well, so the weights are no longer trustworthy as “the effect of this feature”.

I had it twice, deliberately and accidentally. **Deliberately:** my Week 6 interaction term `fuel_bill = litres × price` correlates **0.989** with `litres`, which gave a Hessian condition number of about **16,000** and is why gradient descent stopped converging on weights while still agreeing on predictions. **Accidentally:** the fixed-pair city columns in Week 2 were *perfectly* collinear — identical columns — which is why I dropped them.

I also dropped `estimated_time_minutes` for this reason: it is nearly a copy of `distance_km` and adds collinearity without adding information.

Q26. Explain the bias–variance tradeoff using your own results.

Bias is error from a model too simple to capture the truth; variance is error from a model so flexible it fits the noise. Total error is roughly $\text{bias}^2 + \text{variance} + \text{irreducible noise}$, and you usually trade one against the other.

My clearest example is the parking sub-model. The data has *no signal* — parking is drawn uniformly at random — so all the error is irreducible noise. A linear model scores $R^2 \approx 0.000$ (high bias, no variance: it just predicts the mean). A DecisionTree scores train 0.9888 and test **−1.00** — it drove bias to zero on the training set by memorising noise, and paid for it with enormous variance.

The traffic classifier shows the useful middle. An unrestricted forest memorised the training rows; capping `max_depth` at 4 *added* bias and *reduced* variance, and the test score went up by 0.92 points. That is the tradeoff being resolved in the right direction.

Q27. Why is a train/test split necessary at all? And why 80/20?

Because a model's score on data it was fitted to is not evidence it will work on data it has not seen — a model that memorises scores perfectly and generalises not at all. The test set simulates unseen data.

80/20 is a convention balancing two needs: enough training data to fit well, enough test data for the score to be stable. On 25,000 rows that gives 20,000 train and 5,000 test, which is comfortable on both counts. I fixed `random_state=42` so the split is identical everywhere in the project — otherwise the model comparison tables would not be comparing like with like.

For the classifiers I use a **stratified** split, so each class appears in the same proportion in both halves. Without it a rare class could land almost entirely in one side.

Q28. What is data leakage? Did it happen to you?

Leakage is when information from outside the training data influences the model, making the evaluation optimistic. **Yes, it happened, and I found and fixed it.**

My original code fitted `MinMaxScaler` on the *whole* feature matrix before `train_test_split`. So the test set's minima and maxima influenced the transformation applied to the training data. The effect is small for MinMax on this data, but it is precisely the mistake cross-validation exists to catch.

The fix is to wrap every model in `Pipeline([MinMaxScaler, model])`. Inside a Pipeline, cross-validation re-fits the scaler on each training fold, so nothing from the validation fold ever reaches it.

There is a second, subtler kind I also handled: when chaining sub-models, predictions on their own training rows are *optimistic*. So the training component features come from `cross_val_predict` — out-of-fold — meaning no row's features were produced by a model that had already seen that row. That is stacking done correctly.

Q29. Why sine and cosine for the departure hour?

Because hour is **cyclical**. 23:00 and 00:00 are one hour apart, but as raw integers they are 23 apart — the maximum possible distance. Any model treating the number literally would think late night and early morning are opposite extremes.

Mapping to $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$ places the hours on a circle, so 23:00 and 00:00 end up adjacent. I do the same for month, so December and January are neighbours.

I also add an explicit `is_peak` flag, because sin/cos alone *cannot* express “high between 8–11 *and again* between 17–21” — that is a two-humped shape a single sinusoid cannot produce, so a linear model is handed it directly.

Q30. Why is accuracy alone not a result?

Because it has no meaning without its **baseline**. If 90% of a dataset is one class, a model that always predicts that class scores 90% accuracy while having learned nothing.

So I always report the majority-class baseline beside the accuracy. My traffic classifier scores 56.96% against a 37.66% baseline — a lift of +19.3 points, which is a real if modest result. My original-data classifiers score 50.88% against a 50.88% baseline — a lift of exactly zero, which is no result at all. The two accuracies are similar; the two *results* could not be more different.

Group C — specifics of this project

Q31. What were the five constants you removed, and why did it matter?

`WINDING_FACTOR = 1.2355` (haversine to road km), `TOLL_RATE = 1.301` (₹/km), `DEFAULT_PARKING = 70.0`, `TRAFFIC_MULT` (fuel burn by traffic level), and `MAINTENANCE` (₹/km by vehicle).

It mattered because **only the final addition was a model**. Everything above it was arithmetic, and that arithmetic produced `fuel_consumption_litres` — the regressor's strongest feature at correlation +0.96. So the model was being handed three of the four terms of the cost formula. Each constant is now a fitted sub-model with its own measured error, and every table keeps the constant as its baseline row: a model that cannot beat the thing it replaced has not earned its place.

Q32. Your distance model only improves on the constant by 10.7%. Was it worth it?

Yes, and the honest framing is more interesting than the headline. The constant is nearly **unbiased** — averaged over 840 real routes it is within 0.4% of the truth. That is why it survived review for so long.

But averaged accuracy is not what a user experiences. Route by route the constant is off by **6.3%** typically, and by up to 66% at the extremes. **A user takes one trip, not the average of all of them, and on that trip the errors do not cancel.**

Concretely: on Jaipur → Agra the constant predicts 274 km against a real 238 km — 15% high. On Surat → Bhavnagar it predicts 116 km against a real 339 km, because the Gulf of Khambhat is in the way. The model cannot fix all of that — endpoint coordinates only partly determine route shape — but 10.7% of MAE and 12% of RMSE is real, and it comes with a measured error bar rather than an assertion.

Q33. One of your sub-models failed completely. Why did you keep it in the report?

Because it is a result. I fitted six model families to `parking_cost` and **every one scored test $R^2 \leq 0$** — worse than simply predicting the mean.

That is the correct answer, not a bug. The generator draws parking uniformly from a fixed set, independently of every other column, so nothing in the data predicts it. A model that *appeared* to predict it would be evidence of leakage.

So the application serves the training mean and `/api/metrics` discloses the failure, with a test asserting the disclosure stays there. **A model that correctly discovers a variable is unpredictable is worth more than one tuned until it looks respectable** — and hiding it would have made the other five sub-models less believable, not more.

Q34. What is stacking, and why did you need it?

Stacking is layering models: the outputs of several base models become the input features of a final one. My chain is exactly that — six sub-models feeding a cost regressor.

It needs care for two reasons. **First, the train/serve mismatch:** if the final model is trained on *true* component values and then served *predicted* ones, it has learned to trust inputs that are exact and receives inputs that are not. **Second, optimism:** a sub-model's predictions on its own training rows are better than it will manage in production, so training the final model on those would make it over-trust its inputs.

I handled both. The training component features come from `cross_val_predict` — out-of-fold, so no row's features came from a model that had seen it. And I measured the mismatch rather than assuming: trained-on-truth scored MAE ₹390.56, chain-consistent scored ₹391.16. **The mismatch cost essentially nothing here**, because the sub-model errors are close to zero-mean so the linear coefficients barely change. I ship the chain-consistent version because it is the correct construction, but I report that the mismatch did not bite rather than claiming a win I did not get.

Q35. You said feature engineering beats model complexity, then showed the opposite. Which is it?

Both, in different regimes — and knowing where the boundary is was the point.

When distance and litres are given (Week 6), the only thing a linear model cannot express is the product `litres × price`. Handing it over takes MAE from ₹143.85 to ₹42.60 and beats both ensembles, which had the same information but had to approximate the product.

End-to-end (Week 10), the model must first infer road distance from four coordinates. The engineered columns save ₹67 of MAE; switching to 300 trees saves **₹474**. The ensemble wins by a factor of seven.

The resolution: **feature engineering wins when the missing structure is something you can write down. When the missing structure is a map — coastlines, mountains, the Gulf of Khambhat — no finite set of hand-made interaction terms captures it, and partitioning space is exactly what trees do.**

Q36. Your feature importance shows vehicle type at under 0.1%. So it doesn't matter?

That is the trap, and no — it does matter. Week 4 recovered a real per-km maintenance rate that differs by vehicle: Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 ₹/km. Week 6 showed that dropping those columns made the app quote an *identical* price for an SUV and a hatchback on the same route, which is a bug I have a regression test for.

The rate spread is about 0.27 ₹/km — roughly ₹108 on a 400 km trip. In the end-to-end model that signal sits *underneath* the toll and parking noise, which is hundreds of rupees. **The importance is low because the noise is large, not because the effect is absent.**

This is exactly why the application serves the chained pipeline: pin distance and litres down first, and the vehicle term becomes visible again. It is also a good general warning about impurity importance — it tells you what the model *used*, not what is *true*.

Q37. Why did you collect real data instead of just using the generated dataset?

Because a distance model trained on the generated dataset would be **circular**. That dataset sets `distance_km = haversine × N(1.2355, 0.082)`, so a model fitted to it would just recover the constant 1.2355 plus irreducible noise — and I would be “proving” that a model beats a constant by fitting a model to data the constant generated.

So I fetched **840 real driving distances** from OSRM over a stratified sample of city pairs. That is genuinely observed data, and it is the only observed data in the project. It is what let me discover that the constant is unbiased-but-wrong, and to find the Gulf of Khambhat routes.

I am also explicit about the consequence: I keep a second, deliberately circular dataset-convention distance model, used *only* inside the chained evaluation so the cost comparison is apples-to-apples against a dataset whose costs came from its own distance column. The real-data model is the one that ships.

Q38. Why does your app prefer live routing over your own model?

Because a real answer beats a predicted one whenever it is available. OSRM returns the actual road distance; my model estimates it with MAE 30.67 km. If routing is reachable, using it is strictly better.

The model is the **fallback**, and that is where it earns its place — the previous fallback was the constant, with an unmeasured error. Now the fallback has a known held-out error, and the response reports `distance_source` so a user can see which path was taken.

Q39. What does the winding factor of 3.60 mean? Isn't that an outlier you should have removed?

It is real, and removing it would have been the wrong call. Surat and Bhavnagar sit on opposite shores of the **Gulf of Khambhat**: 94 km of straight line, but you have to drive around the head of the gulf, which is 339 km of road.

That point, and the 1.04 factor of Moradabad → Chanduasi at the other extreme, are the *entire argument* for using a model instead of a constant — no single multiplier can serve both. Deleting them would have removed the evidence for my own conclusion.

It also explains which model won: a **tree ensemble**, because it can isolate a region of the map. A linear fit gets dragged by such points rather than representing them.

Q40. How do you choose between two models that score almost the same?

Three filters, in order, and it is written down so it is reproducible.

1. **Test-error band** — keep models within 2% of the best RMSE (or 0.5 accuracy points). Crucially, closeness is judged on the *error*, never on R^2 .
2. **One-standard-error rule** — of those, keep every model whose CV mean is within one standard error ($sd/\sqrt{k}$) of the best. This is the standard rule from Hastie, Tibshirani & Friedman §7.10: a difference smaller than the noise in the CV estimate is not a real difference.
3. **Simplicity** — take the simplest survivor. Here that is not only Occam's razor: a linear model keeps my from-scratch gradient descent applicable, which no ensemble does.

I know the first filter matters because I got it wrong first. My original rule used " R^2 within 0.5% relative", and near $R^2 = 0.99$ that band swallowed a model with 15% more RMSE — it picked a Ridge fit at 53.7 km over a random forest at 46.6 km.

Q41. Why does your cost-band classifier score 85% when the traffic one only scores 57%? Isn't the 85% the better model?

No — the 57% is the better *result*, and I would volunteer that.

`cost_band` is a quartile split of ₹/km, and my Week 6 regressor already predicts ₹/km to within ₹42. So the classifier is being asked to discretise a quantity that is already known accurately. Its 85% is **high by construction**; it earns its place because the interface needs the label, not because it demonstrates difficult learning.

`traffic_level` is genuinely non-trivial: departure hour only partly determines traffic, so the ceiling is low. 57% against a 37.66% baseline is a lift of +19.3 points on a hard problem. **A classifier claiming 95% there would be evidence of leakage, not skill.**

Q42. What does your confusion matrix tell you beyond the accuracy?

How the model fails, which accuracy cannot show. For the traffic classifier, errors are concentrated between **adjacent** levels — Low confused with Medium, Medium with High — and it is most accurate at the extremes.

That is the right failure mode for an ordered target: it has learned the rush-hour structure and is uncertain in the middle, which is where the data genuinely is ambiguous. A model that confused Low with High would be a much more worrying 57%.

On the cost-band matrix the same reading applies: Excellent and Poor are classified best (F1 0.895 and 0.919), the two middle bands worst (0.816 and 0.801), because the boundary between "Good" and "Average" is an arbitrary quartile cut.

Q43. Why 5-fold and not 10-fold, or leave-one-out?

It is a bias–variance–cost tradeoff on the *estimate*. More folds means more training data per fold, so less pessimistic bias, but more models to fit and a higher-variance estimate. Leave-one-out is the extreme: nearly unbiased, expensive, and high variance.

5-fold is the standard compromise and the checklist's default. With 20,000 training rows each fold still has 16,000 rows to train on, which is plenty — and my measured CV standard deviations of $1e-5$ confirm the estimate is stable, so more folds would buy nothing here. Fitting 7 models $\times$ 5 folds $\times$ several targets was already the bulk of my compute time.

Q44. What is the difference between GridSearchCV and RandomizedSearchCV, and when do you use each?

Grid search tries **every** combination in a specified grid — exhaustive and guaranteed to find the grid's best, but the cost is the product of all the axes. Randomized search samples a fixed number of random combinations from distributions — it covers a larger space for a fixed budget, and it does not waste effort on axes that turn out not to matter.

I used both, matched to the problem. Grid for Ridge α (one axis, 8 values — exhaustive is trivial and I wanted the whole curve to inspect). Grid for the traffic classifier (3 axes, 18 combinations — still cheap). Randomized for RandomForest, because the space is 4 axes with $4 \times 4 \times 4 \times 3 = 192$ combinations and 12 well-chosen draws tell me what I need for a sixteenth of the compute.

Q45. How long does everything take to run, and what is the biggest cost?

The OSRM data collection is the wall-clock bottleneck at about **6 minutes** for 840 requests, rate-limited to 0.4 s apart to be a polite client on a free public server. It writes each row to disk as it goes, so a rate-limited run can resume.

Training the six sub-models takes about 4–5 minutes, the Task 5 sweep about **323 seconds**, and the end-to-end benchmark about 2 minutes. The most expensive single step is the RandomizedSearchCV: 12 candidates $\times$ 5 folds of a 300-tree forest on 20,000 rows, about 86 seconds even parallelised across 20 cores.

Group D — harder and follow-up questions

Q46. Your model was trained on generated data. How would it perform on real trips?

Worse, and I can be specific about where. The **cost structure** should transfer reasonably — fuel = litres $\times$ price is physics, not an artefact. The **distance** model should transfer well, because it is trained on real OSRM data.

Three things would degrade. The **traffic** accuracy would almost certainly fall, because my 57% measures an injected rush-hour profile rather than real Indian roads — I say so explicitly. **Toll** would change most: real Indian tolls are per-plaza, not a smooth per-km rate, so the linear form is wrong even though it fits the generated data.

Parking is currently unpredictable by construction; in reality it correlates with city size and destination type, so it would actually become *more* predictable with real data.

The honest summary: this project's methodology would transfer; its coefficients would need refitting; and its reported accuracy is an upper bound.

Q47. What would you do differently if you started again?

Four things. **Test the data-generating process first** — I reached Week 4 before asking whether the data was synthetic, and that question should have come in Week 1 when I noticed zero missing values and zero duplicates in 1,140 rows.

Split before any data-dependent transformation. My Week 2 outlier removal and scaling both happen before the split. I fixed the scaling with Pipelines; the outlier filter is still applied to the whole dataset, which is a real if minor leak.

Collect real data earlier. The 840 OSRM distances turned out to be the most valuable data in the project and they took six minutes to gather.

Report the served metric from the start. The ₹39.66 error bar sat next to predictions for weeks before I noticed no real prediction could achieve it.

Q48. How would you improve the accuracy from here?

The error budget answers this precisely, which is why I built it. Of the ₹596 RMSE, toll ($\pm ₹219$) and parking ($\pm ₹50$) are **irreducible** — drawn from random distributions by construction. No model can improve them.

The only improvable term is the **fuel bill ($\pm ₹479$)**, and it is improvable because it inherits the distance error. So the highest-value work is a better distance model: more real OSRM routes (840 is a small training set for a geographic problem), terrain and road-class features rather than just endpoint coordinates, or simply caching real routing more aggressively so the model is needed less often.

Everything else — a bigger ensemble, more tuning, more features on the cost regressor — would be working on the ₹67 that is left, and my RandomizedSearchCV already showed there is nothing there.

Q49. Why is there no neural network?

Because nothing in this problem calls for one, and I can say what would.

The cost target is a linear combination of terms plus one known product. A linear model with the right features fits it to R^2 0.9997 and I can read every coefficient. A neural network would be slower to train, need far more data to justify its parameter count, lose the interpretability that makes the coefficients meaningful, and — on the evidence of the ensembles — almost certainly score worse.

Where one *would* help is the end-to-end problem: predicting cost from raw coordinates is a genuinely non-linear geographic function, and a network could learn a spatial embedding that trees approximate with axis-aligned splits. With substantially more real routing data, that would be the natural next experiment.

Q50. Your test set is 5,000 rows from the same generated distribution. Is that a fair test?

It is a fair test of *generalisation within that distribution*, and nothing more — which is the right way to state it. It answers “does the model work on trips it has not seen?” but not “does the model work on trips from a different distribution?”

A stronger design would hold out entire *routes* or entire *regions*, not random rows. Because I sample 5 trips per route, the same city pair can appear in both halves — so the model may benefit from having seen that route's geography. For the cost regressor this matters little (cost is a function of the numeric features, not of route identity), but for the distance model it would matter a lot, which is why that model is trained and tested on a separate real dataset with its own split.

Route-level or region-level cross-validation is the improvement I would make.

Q51. What happens if a user asks for a 3,000 km trip?

The prediction is returned *with an explicit warning that it is extrapolating*. The models were trained on 50–1,500 km, and beyond that range they have no evidence.

I chose to warn rather than refuse because a refusal is less useful than a caveated number, and less honest than silently guessing — which is the third option and the worst one. The warning names the trained range so the user can judge how far outside it they are.

Q52. Why does the same route cost differently at 3am and 9am?

Through traffic, and only through traffic. The traffic classifier infers the likely level from the departure hour; a higher traffic level multiplies fuel consumption (Low 0.944, Medium 1.045, High 1.165); more litres means a bigger fuel bill.

Distance, toll and parking are unaffected by departure time, which the API states in its response note. On Jaipur → Agra the full 24-hour spread is about ₹177. The departure-sweep endpoint charts this, using the classifier's *probabilities* rather than its most likely label — taking the argmax would collapse 24 hours onto 3 distinct values.

Q53. You have two accuracy numbers in your API. Isn't that confusing for a user?

The API leads with one and labels it `served`: R^2 0.9645, MAE ₹391. That is the number a user sees beside their estimate. The larger figure is available as `fed_true_components` with a note saying it answers a different question, plus an explicit `honesty_gap` field.

Reporting only the flattering number would be misleading; reporting only the honest one would throw away a genuine result about the model and the dataset. Labelling both, with the gap named, is the version that is true and complete — and there is a regression test asserting the served figure is the smaller of the two, so the flattering number cannot creep back into the headline.

Q54. Why did you write the report generator instead of typing the numbers?

Because the report quotes roughly two hundred numbers. Typed by hand it would start with at least one transcription error and go stale the first time I retrained anything — and a report whose numbers disagree with the code is worse than no report.

So `TASK5.md` and the results sections of this document are generated from the same saved JSON that the notebook and the API read. Regenerating after a retrain is one command, and the four artefacts cannot contradict each other.

Q55. How do you know your results are reproducible?

Every random operation is seeded: `random_state=42` for splits and models, a fixed seed for the route sampling and the bootstrap. I verified it in practice — after re-running the whole pipeline to fix the model file sizes, every metric came back **bit-identical** (R^2 0.964542, MAE ₹391.16, distance MAE 30.67 km), which is why the notebook outputs and the generated report stayed valid without re-execution.

Beyond seeding: the build order is documented, each script says what it needs and what it produces, the 65 tests pin the behaviour, and the notebook chapters are generated by scripts so they can be rebuilt rather than hand-maintained.

Q56. What is the single most interesting thing you learned?

That **a near-perfect score can be a statement about the question, not the model**. R^2 0.9997 was not a mistake or a bug — it was a correct measurement of something I had not intended to measure. The model really could total a bill given three of its four terms; it just could not do the thing a user needed.

Finding that required knowing where the data came from, which is why Week 4 mattered so much. It changed how I report every number in the project, and it is the reason I now instinctively ask “what exactly is this model being given?” before believing a score.

Q57. Which part of this project are you least confident about?

The toll sub-model. It does not beat the constant it replaced (R^2 0.7949 against 0.7950), and I keep it mainly because it is *fitted and measured* rather than asserted. On the generated data that is defensible, since the per-trip spread is injected noise. But real Indian tolls are charged per plaza, not as a smooth per-kilometre rate, so I suspect the whole functional form is wrong and only looks acceptable because the synthetic data was generated that way. With real toll data I would expect to need a different model entirely — probably one keyed on which national highways a route uses.

Q58. If I gave you one more week, what would you do?

Collect more real routing data — several thousand OSRM routes instead of 840 — and add road-class and terrain features to the distance model, because the error budget says that is the only term worth improving. Then switch to **route-level cross-validation** so no city pair appears in both halves of the split, which is the weakest part of my current evaluation.

After that, replace the per-km toll model with something keyed on the highways a route actually uses, and re-run the whole Task 5 sweep to see which conclusions survive real data. I would expect the traffic accuracy to fall and the parking model to become viable.

Q59. Explain your project to someone who knows no machine learning.

"You tell it where you are driving from and to, what car you have, and when you are leaving. It tells you roughly what the trip will cost, and it shows its working — how much is fuel, how much is tolls, how much is wear on the car."

"It learned the relationship from 25,000 example trips. The interesting part is that it is honest about what it does not know: it says the estimate is give-or-take about ₹400, and most of that is tolls and parking, which nobody can predict in advance for a specific trip."

Q60. Anything you want to add?

One thing: the project contains as many negative results as positive ones, and they were kept deliberately. A parking model that could not beat the mean. A tuning search that found no improvement. A distance model that only improved things by 10.7%. A train/serve mismatch that turned out not to matter. A finding from Week 6 that reversed in Week 10.

Each of those was easier to delete than to explain. Keeping them is what makes the positive results believable — and every one of them taught me more than the successes did.

Glossary — every technical term used in this project

Alphabetical. If an examiner uses one of these words, this is the definition to give, with the project's own example attached where there is one.

Term	Definition, and where it appears here
AdaBoost	Adaptive Boosting. Trains weak learners (usually shallow stumps) sequentially, re-weighting the rows the previous learner got wrong. Consistently the weakest ensemble in this project, because chasing the noisiest rows is the wrong instinct on a smooth additive target.
Bagging	Bootstrap Aggregating. Trains many models in parallel on bootstrap samples of the rows and averages them, reducing <i>variance</i> . RandomForest is bagging plus random feature subsets per split.
Baseline	The score to beat before a model means anything. For classification, the majority-class rate; for a replaced constant, the constant's own error. Every table in this project carries one.
Bias (statistical)	Systematic error — being wrong in the same direction on average. The winding-factor constant is nearly <i>unbiased</i> (−0.4%) yet wrong by 6.3% per route, which is the distinction Part 12 turns on.
Bias–variance tradeoff	Error decomposes into bias^2 (model too simple) + variance (model fits noise) + irreducible noise. Capping the traffic classifier's <code>max_depth</code> at 4 added bias, cut variance, and improved the test score.
Bootstrap	Resampling the data <i>with replacement</i> to estimate the variability of a statistic. Used here as the alternative to k-fold: 200 resamples gave a 95% interval of width $3e-6$.
Boosting	Training models sequentially, each correcting its predecessors' errors. Attacks <i>bias</i> , unlike bagging which attacks variance.
Chained model / pipeline	Feeding one model's output into another's input. This project chains six sub-models into a cost regressor. See also <i>stacking</i> .
Coefficient / weight	The number a linear model multiplies a feature by. Because features here are scaled to 0–1, each weight reads as “rupees of change as this feature goes from its minimum to its maximum”.
Condition number κ	Ratio of largest to smallest eigenvalue of the Hessian; how “stretched” the loss surface is. At $\kappa \approx 16,000$ the surface is a long narrow valley, which is why gradient descent stopped converging on weights in Week 8.
Confusion matrix	A table of actual class against predicted class. Shows <i>how</i> a classifier fails, not just how often — this project's traffic errors concentrate between adjacent levels, which is the right failure mode for an ordered target.
Convex function	A function with a single minimum and no local minima. MSE for linear regression is convex, which is <i>why</i> gradient descent and the normal equations must reach the same answer.
Correlation (Pearson r)	Strength and direction of a linear relationship, from −1 to +1. <code>fuel_consumption_litres</code> correlates +0.9606 with the target; <code>passengers</code> −0.0468.
Cross-validation (k-fold)	Split the training data into k parts, train on k−1 and validate on the rest, k times. Gives a mean and, more usefully, a standard deviation that reveals instability.
<code>cross_val_predict</code>	Produces out-of-fold predictions: each row's prediction comes from a model that did <i>not</i> see that row. Essential for building stacking features without optimism.
Data leakage	Information from outside the training data influencing the model, making evaluation optimistic. This project had it — a scaler fitted before the split — and fixed it with Pipelines.
DecisionTree	Recursively splits the feature space on thresholds. Grown to purity it memorises the training set (train $R^2 = 1.0000$), which is why it is the project's clearest overfitting example.
Design matrix	The <code>X</code> matrix of features, rows = samples, columns = features.
Dummy variable trap	Keeping all one-hot columns of a group makes them sum to 1 in every row, which is collinear with the intercept and makes the matrix singular. Avoided with <code>drop_first=True</code> .

Epoch	One full pass of gradient descent over the training data. This project's six-feature fit converged at epoch 14,317 of a 50,000 budget.
F1 score	Harmonic mean of precision and recall, $2PR/(P+R)$. Penalises a model that is good at one and bad at the other.
Feature engineering	Constructing new input columns from existing ones. The project's largest single accuracy gain: adding <code>litres × price cut</code> MAE from ₹143.85 to ₹42.60.
Feature importance	How much each feature contributed. For trees, impurity reduction; for a scaled linear model, <code> coefficient </code> . It reports what the model <i>used</i> , not what is <i>true</i> — see the vehicle one-hot trap in Part 12.7.
GradientBoosting	Boosting where each new tree is fitted to the residuals of the ensemble so far. Won the traffic classification here.
Gradient descent	Iterative optimisation: compute the loss, compute its gradient, step in the opposite direction. Implemented from scratch in NumPy in Week 3.
GridSearchCV	Exhaustive search over every combination in a parameter grid, scored by cross-validation. Used for Ridge α and the traffic classifier.
Haversine	Great-circle (straight-line) distance between two points on a sphere. The starting point for road distance, which is always longer.
Hessian	Matrix of second derivatives of the loss; describes the curvature of the loss surface. Its condition number is what diagnosed the Week 8 problem.
Hyperparameter	A setting chosen <i>before</i> training, not learned from data — learning rate, number of trees, <code>max_depth</code> , Ridge α . Contrast with <i>parameters</i> (weights), which are learned.
Interaction term	A feature formed by multiplying two others, letting an additive model express a product. This project's central technique — used for the fuel bill, for the litres model's traffic multipliers, and for the mileage model's nine cells.
IQR method	Outlier rule: drop points outside <code>[Q1 - 1.5×IQR, Q3 + 1.5×IQR]</code> . Removed 11 of 1,140 rows in Week 2.
Irreducible error	Error no model can remove because the data itself is random. Here: toll noise ($\pm ₹219$) and parking noise ($\pm ₹50$), which together with the fuel-bill term account for ₹529 of the ₹596 RMSE.
L1 / L2 regularisation	L1 (Lasso) penalises $\sum w $ and can force weights to exactly zero, so it selects features. L2 (Ridge) penalises $\sum w^2$ and shrinks weights smoothly without zeroing them.
Learning rate	Step size in gradient descent. Too small and it never arrives ($lr = 0.001$ left weights 2,025 away); too large and it diverges (above $lr \approx 0.44$ here). This project uses 0.2.
LinearRegression	Fits $\hat{y} = Xw + b$ by minimising squared error. sklearn solves it in closed form via the normal equations. The model this project ships.
LogisticRegression	Linear model for <i>classification</i> — fits a linear function then squashes it through a sigmoid/softmax into class probabilities. Best model on <code>cost_band</code> .
Loss function	The quantity being minimised. Here MSE, $(1/n)\sum(\hat{y} - y)^2$.
Macro vs weighted averaging	Macro weights every class equally; weighted weights by class frequency. Macro is reported here because weighted averaging can hide a class the model never predicts — demonstrated by macro F1 0.2248 at 50.88% accuracy on the original data.
MAE	Mean absolute error, $(1/n)\sum y - \hat{y} $. The number to tell a user. Served value here: ₹391.16.
MinMaxScaler	Rescales each feature to 0–1 via $(x - \min)/(\max - \min)$.

MSE / RMSE	Mean squared error and its square root. RMSE reads in the target's units and penalises large errors more heavily than MAE.
Multicollinearity	Features strongly linearly related to each other. The fit stays valid but individual coefficients become unstable. Here: <code>corr(litres, fuel_bill) = 0.989</code> .
Multiple linear regression	Linear regression with more than one feature: $\hat{y} = w_1X_1 + \dots + w_kX_k + c$.
Normal equations	The closed-form solution $w = (X^TX)^{-1}X^Ty$. Exact, one step, and indifferent to the condition number — which is why this project ships it instead of gradient descent.
One-hot encoding	Turning a categorical column into one binary column per category, asserting no ordering between them.
One-standard-error rule	Model-selection rule from Hastie, Tibshirani & Friedman §7.10: among models within one standard error of the best CV score, choose the simplest, because smaller differences are not real.
Out-of-fold prediction	A prediction for a row made by a model trained without that row. The correct way to build stacking features.
Overfitting	Learning the training data's noise; good train score, poor test score. Seen here on <code>cost_band</code> (gap 0.21) and the toll sub-model (gap 0.37).
Pipeline (sklearn)	Chains transformers and a final estimator into one object, so cross-validation re-fits the transformers <i>per fold</i> . The fix for this project's scaler leak.
Precision	<code>TP / (TP + FP)</code> — of the items predicted positive, how many were. “When it says High traffic, how often is it right?”
R²	Coefficient of determination, $1 - \text{RSS} / \text{TSS}$. Fraction of variance explained. Compresses meaningful differences when the target is nearly deterministic.
RandomForest	Bagged decision trees with random feature subsets per split. Strongest ensemble here; wins where the relationship is genuinely non-linear.
RandomizedSearchCV	Samples a fixed number of random hyperparameter combinations instead of trying all. Covers a bigger space for a fixed budget.
Rank deficiency	When columns of the design matrix are linearly dependent, so infinitely many weight vectors give identical predictions. Why the fixed-pair city one-hots were dropped in Week 3.
Recall	<code>TP / (TP + FN)</code> — of the items that <i>are</i> positive, how many were found.
Regularisation	Penalising weight size to reduce variance. Rejected here after measuring that it cost six times the error.
Residual	$y - \hat{y}$. Studying residuals is how Week 4 recovered the generating formula.
Ridge regression	Linear regression with an L2 penalty: $w = (X^TX + \alpha I)^{-1}X^Ty$.
RSS	Residual sum of squares, $\sum (y - \hat{y})^2$. A sum, so comparable only within one split.
Stacking	Using base models' outputs as a meta-model's inputs. This project's chained pipeline, built with out-of-fold predictions.
Stratified split	A split preserving each class's proportion in both halves. Used for all classification splits here.
Target / label	The column being predicted. Here <code>total_trip_cost</code> .
Train/serve mismatch	Training a model on inputs of a different quality from what it receives in production. Measured explicitly here — and it cost essentially nothing, because the sub-model errors are zero-mean.
Underfitting	Model too simple, or features too weak, to capture the relationship; poor on both splits. All five classifiers on the original dataset's traffic target.

Variance (of a model)	Sensitivity to which particular training rows were seen. High-variance models score well on train, poorly on test.
Winding factor	This project's term for road distance ÷ straight-line distance. Observed mean 1.2520, range 1.0437 to 3.6015 across 840 real routes — the variation a single constant cannot express.

How to reproduce every number in this report

17.1 Just run the application

```
pip install -r requirements.txt
uvicorn app:app                # -> http://127.0.0.1:8000
```

Everything needed is committed: the trained models, the datasets and the built frontend.

17.2 Rebuild everything from scratch, in order

```
python scripts/build_cities.py           # 3,739 Indian cities with real lat/lon (GeoNames)
python scripts/build_fuel_prices.py       # per-state fuel prices
python scripts/build_wide_dataset.py      # 25,000 trips over real geography
python scripts/fetch_real_distances.py    # 840 REAL road distances from OSRM (~6 min)

python scripts/train_models.py            # -> models/model.joblib (Week 6 regressor)
python scripts/train_pipeline.py          # -> models/pipeline.joblib (the six sub-models)
python scripts/train_end_to_end.py        # -> the honesty benchmark

python scripts/evaluate_models.py         # -> data/task5_evaluation.json (Task 5, ~5 min)
python scripts/build_task5_doc.py         # -> TASK5.md

python scripts/build_notebook_chapters.py # notebook Weeks 4-8
python scripts/build_task5_chapters.py    # notebook Weeks 9-10
python scripts/execute_new_chapters.py    # execute the appended cells, keep their outputs

pytest -q                                # 65 tests
```

Order matters for the notebook steps. `build_notebook_chapters.py` rebuilds everything from Week 4 onwards, so it *wipes* Weeks 9–10 and they must be re-appended after it. `execute_new_chapters.py` executes only the appended cells — running the whole notebook would redo the 60,000-epoch gradient descent for no reason.

17.3 Which file produces which number

Number in this report	Produced by
Weeks 1–3 — all figures	RoadTripCost.ipynb cells 0–74, executed
Week 4 formula recovery	notebook cells 75–83
Week 5 dataset statistics	scripts/build_wide_dataset.py
Weeks 6–8 tables	notebook cells 88–106
Part 11 (Week 9 / Task 5)	data/task5_evaluation.json via scripts/evaluate_models.py
Part 12 sub-model tables	data/pipeline_report.json via scripts/train_pipeline.py
Part 12 end-to-end tables	data/end_to_end_report.json via scripts/train_end_to_end.py
The 840 real distances	data/real_distances.csv via scripts/fetch_real_distances.py
Parts 11–12 of this PDF	scripts/build_report_tables.py and scripts/build_report_week10.py — generated from the JSON, never typed

17.4 Reproducibility guarantees

- **Every random operation is seeded.** `random_state=42` for all splits and models; a fixed seed for route sampling and for the bootstrap.
- **Verified in practice.** After re-running the whole pipeline to fix the model file sizes, every metric returned *bit-identical* — R^2 0.964542, MAE ₹391.16, distance MAE 30.67 km — which is why the notebook outputs and the generated documents remained valid without re-execution.
- **One scoring module.** All metrics come from `scripts/evaluation.py`, so the notebook, `TASK5.md`, this PDF and the API cannot disagree.
- **65 tests** pin the behaviour, including the error budget and the honest reporting.

Closing note

If only one thing from this report is remembered going into a viva, make it this: the project reports **R^2 0.9645 and MAE ₹391** as its accuracy, not the R^2 0.9997 that appears in Weeks 6 to 8. The larger number is real but was measured with the model holding three of the four terms of its own answer. Knowing the difference — and being able to explain *why* the difference exists — is what the ten weeks were actually for.